Gimel Foundation

Gimel Foundation gGmbH i.G.
GiFo-Request for Comments: 0140
Obsoletes: -
Category: Standards Track
Version: 1.1

G. Wehberg, C. Pinkel and the Architecture Working Group of Gimel Foundation
17. April 2026

G-AGT Integration Profile

Abstract

Today, there are many attempts to govern AI through new toolkits and solutions. Many of these represent a platform-bound enforcement of policies, initiated by hooks. These attempts, though, are not necessarily sufficient to meet cybersecurity requirements and regulations like the EU AI Act amongst others, which requires to ensure that AI systems developed or used within the EU are safe, trustworthy, and under control (see also GiFo-RFC 0130). Next to a platform bound-enforcement of policies, it requires a credential-bound enforcement of the specific authority an AI system is supposed to have to act, decide and enter transactions. To integrate both, this is what this Request for Comment 0140 is about.

The G-AGT integration profile defines how a typical Agent Governance Toolkit (AGT) - exemplified by Microsoft's AGT and informed by publicly available operational governance solutions - integrates with the GAuth authorization architecture (GiFo-RFCs 0110, 0111, 0115, 0116, 0117, and 0118). G-AGT specifies two integration scenarios and acknowledges a dynamic third:

- **Scenario “AGT Engine”**: AGT serves as an access control vehicle within GAuth's PEP extension point - analogous to how an OAuth engine (e.g., Ory Hydra) serves as the authorization vehicle in GAuth's Type A adapter slot.
- **Scenario “AGT Bridge”**: Full AGT runs standalone with its native capabilities active (policy evaluation, execution rings, trust scoring, kill switch, circuit breakers). AGT calls into GAuth's PEP for credential-bound delegation enforcement decisions.
- “**Dynamic” Scenario**: In a dynamic future, AGT implementations (or successors) could integrate the full GAuth specification suite (RFCs 0110-0118). G-AGT (RFC 0140) retains its purpose as the normative integration profile, the exclusive gateway to Gimel's proprietary services via Type C adapter interface contracts, and the conformance authority that defines what "G-AGT compliant" means.

Under all scenarios, the GAuth PEP (Policy Enforcement Point) remains the authoritative governance control plane. G-AGT positions GAuth as the authority and policy layer that AGT's runtime hooks call into.

This specification excludes AI-enabled governance, web3 integration as well as DNA_based identities and PQC associated, consistently with all other GiFo-RFCs. All trust scoring, policy evaluation, and governance functions defined herein are deterministic and rule-based. AI/ML-enhanced governance capabilities are available exclusively through proprietary Type C adapters (Slot 5) under separate license.

Status of This Memo

This is a Gimel Foundation Standards Track document.

This document is a product of the Gimel Foundation (GiFo). It represents the current consensus of the Gimel Foundation community. It has performed review and has been approved for publication.

Information about the status of this document, any errata, and how to provide feedback on it may be obtained at https://gimelfoundation.com or https://github.com/Gimel-Foundation.

Legal notice

the “Dynamic” Scenario - subject to the Exclusions stated in §1.1 of this RFC 0140. Implementations of G-AGT Must refer to the Apache 2.0 license granted by Gimel Foundation and Must Not integrate Exclusions (as per the Scope statement of this Request for Comment). Defined Exclusions Must refer to separate license conditions.

Implementations of G-AGT Must also refer to the licenses of the building-block standards: the Apache 2.0 license of OAuth and OpenID Connect, the MIT license of MCP, and the licenses of any operational governance component integrated via the AGT interfaces defined herein.

Gimel Foundation has leveraged the Microsoft AGT solution as a starting point (amongst others), to turn code into specs. As a backup, we take the MIT license of the Microsoft AGT implementation into account and ask all users of our G-AGT specs to do so.

Any proprietary solutions (Exclusions) such as AI-enabled governance, Web3 integration, as well as DNA-based identities and PQC associated, are not in scope of this specification. They do not fall under this open-source license but are subject to proprietary licensing by Gimel Foundation or Gimel Technologies, respectively. Consistently with GiFo-RFCs 0110, 0111, 0115, 0116, 0117, and 0118, the use of AI, machine learning, or any non-deterministic computational method for governance purposes, evaluation, trust scoring, policy generation, risk assessment, or compliance assessment, web3 integrations, DNA-based identities, etc. within the GAuth adapter slot system is excluded from this open specification and reserved for proprietary licensing.

Notational Conventions

The key words "Must", "Must Not", "Required", "Shall", "Shall Not", "Should", "Should Not", "Recommended", "May", and "Optional" in the following specification are to be interpreted as described in IETF` s RFC 2119.

* * *

Table of Content

1. SCOPE

G-AGT concerns the technical field of AI governance and specifically the integration of AGT-based operational governance engines within the GAuth authorization architecture. This specification defines three integration scenarios along a spectrum of integration depth:

(i) Scenario "AGT Engine" (Access Control Vehicle)

In this scenario, AGT serves as an access control vehicle within GAuth's PEP Phase 2 extension point (§4.2). AGT provides only the vehicle function: the middleware hook (`PreToolUse`) that connects the agent runtime to GAuth's governance pipeline. GAuth absorbs and implements AGT's core access control primitives - deterministic policy evaluation, execution rings, trust scoring, kill switch, and circuit breakers - natively within its own platform. AGT's native implementations of these capabilities are dormant (§4.4). This parallels how Ory Hydra serves as the OIDC vehicle in GAuth's Type A adapter slot: Hydra provides the protocol vehicle, GAuth orchestrates authorization around it. AGT's runtime hooks (e.g., `PreToolUse`) call GAuth's PEP with verb+resource and receive enforcement decisions with mandate constraints.

(ii) Scenario "AGT Bridge" (Full Operational Governance)

In this scenario, AGT runs standalone with its complete connector breadth - tool governance, memory management, agent marketplace, inter-agent communication -

and all native AGT capabilities active (policy evaluation, execution rings, trust scoring, kill switch, circuit breakers). AGT calls into GAuth's PEP for credential-bound delegation enforcement decisions via a JSON-to-YAML policy translation bridge. GAuth provides credential-bound delegation enforcement. AGT owns platform-bound access control, natively, applying its own operational policies after receiving the PEP decision. Organizations that need AGT's broader connector ecosystem use this scenario.

(iii) Dynamic Scenario — AGT as Full GAuth Implementation

In a dynamic future, AGT (or successors) could implement the full GAuth specification suite (RFCs 0110 - 0118) - the 16-check PEP pipeline, PoA credential scheme, mandate lifecycle, delegation chains, budget enforcement, etc. This is permissible and welcomed under the Apache 2.0 / MPL 2.0 licenses, provided Gimel Foundation's license terms and legal provisions (GiFo-RFCs 0080, 0090, 0100) are properly respected. In this case, G-AGT (RFC 0140) retains its purpose through: (a) being the normative integration profile that defines how delegation and access control governance interoperate, (b) the Type C adapter interface contracts (§8) - the exclusive gateway to Gimel's proprietary services (AI governance, Web3, DNA/PQC), and (c) the conformance requirements (§10) that define what "G-AGT compliant" means.

This specification defines:

- The AGT access control engine requirements (§4.3), specifying the interfaces and behaviours any compliant AGT engine Must provide to serve as a vehicle within GAuth's PEP Phase 2 extension point (Scenario i) or as a standalone engine calling into GAuth (Scenario ii).
- The PEP Phase 2 extension point (§4.2) — the architectural mechanism by which GAuth's PEP invokes an AGT engine for access control evaluation, distinct from the connector slot model (Type A/B/C) and not subject to tariff gating or sealed manifest lifecycle.
- The orchestrated evaluation model (§5) by which GAuth's PEP invokes AGT as an access control engine (Scenario i) or is invoked by AGT via runtime hooks (Scenario ii), maintaining GAuth as the authoritative governance control plane in both cases, with budget deferral transactional semantics (§5.2) for stateful PEP mode.
- Credential integration and policy translation mechanisms (§6) — native JSON-based PoA mapping for Scenario i), and JSON-to-YAML translation bridge for Scenario ii), using the actual GAuth verb taxonomy (`urn:gauth:verb:{domain}:{category}:{action}`).
- Governance profile alignment (§7) between GAuth's five governance profiles and AGT execution privilege levels, incorporating the 5-tier trust boundary model (§4.3.3).

- Trust score and confidence score separation (§4.3.3, §7.4) — the trust score (AGT domain, deterministic, 5-tier) and confidence score (GAuth domain, AI-enhanced, Type C Slot 5) are architecturally distinct and independently preserved.
- Type C adapter interface contracts (§8) that enable optional connection to proprietary governance services — the durable differentiator across all scenarios including the Dynamic Scenario.
- A unified audit trail format (§9) where GAuth's native audit is authoritative, AGT-specific operational telemetry is merged as supplementary data, and both trust score and confidence score are preserved independently.
- Dormancy rules (§4.4) for AGT features that are redundant with GAuth's native capabilities, with scenario-specific scope (§4.4.1).

1.1 Exclusions

The following are explicitly out of scope for this specification, consistently with GiFo-RFCs 0110, 0111, 0115, 0116, 0117, and 0118:

- **AI-Enabled Governance** - AI/ML-based policy generation, risk scoring, compliance assessment, adaptive trust scoring, learning-loop behavioural analysis, or any non-deterministic computational method for governance evaluation. Available exclusively as a proprietary Type C adapter (Slot 5) under separate license from the Gimel Foundation.
- **Confidence Scoring** - The confidence scores (0.0–1.0 float) used for implied authority override and non-deterministic threat detection are produced exclusively by the proprietary GovernanceAdapter (Slot 5). It is architecturally distinct from the deterministic trust score defined in §4.3.3.
- **Web3 Integration** - Blockchain, DLT, decentralized identity, or token-gated access control. Available as a proprietary Type C adapter (Slot 6) under separate license.
- **DNA-Based Identities / Post-Quantum Cryptography** - Biometric identity from genomic data, PQC algorithms (CRYSTALS-Kyber, CRYSTALS-Dilithium, FALCON, SPHINCS+, etc.). Available as a proprietary Type C adapter (Slot 7) under separate license.
- **Proprietary Type C adapter implementations** - This specification defines the open interface contracts for Type C adapter slots. The sealed implementations are subject to proprietary licensing by Gimel Foundation or Gimel Technologies.
- SDK implementations - Code-level SDK implementations of G-AGT are defined in separate specifications (future GiFo-RFCs).

1.2 Relationship to Existing Specifications

- **GiFo-RFC 0110**: Protocol engine architecture model; P*P pattern (PEP, PDP, PAP, PIP, PVP). G-AGT integrates AGT as an engine within this architecture. AGT's policy evaluation architecture follows the same P*P decomposition formalized by RFC 0110 — Policy Evaluator (PDP), policy document management (PAP), PreToolUse enforcement hook (PEP), context resolution (PIP) — establishing RFC 0110 as prior art for the architectural pattern that AGT applies to the operational governance domain.
- **GiFo-RFC 0111**: GAuth authorization framework; roles, flows, delegation. G-AGT presumes a GAuth-compliant authorization server as the authoritative governance control plane.
- **GiFo-RFC 0115**: Power-of-Attorney Credential Definition; three-layer capability model, governance profiles, delegation matrix. G-AGT maps these into AGT policy contexts.
- **GiFo-RFC 0116**: GAuth Interoperability; Extended Token format (JWT/W3C VC), PoA credential schema, OAuth engine integration (Type A adapter pattern). G-AGT follows the same vehicle integration pattern for AGT Engine (Scenario i), but via the PEP Phase 2 extension point rather than the connector slot model.
- **GiFo-RFC 0117**: GAuth PEP Interface; 16-check enforcement pipeline, enforcement modes, HTTP binding. The PEP orchestrates AGT evaluation via the Phase 2 extension point (Scenario i) or is called by AGT's runtime hooks (Scenario ii). Appendix D defines the normative verb taxonomy used for AGT verb mapping.
- **GiFo-RFC 0118**: GAuth Management API; mandate lifecycle, budget, delegation, governance profiles. G-AGT relies on this for mandate provisioning. Budget deferral semantics (§5.2) extend the PEP's interaction with RFC 0118 budget operations.
- **IETF RFC 2119**: Requirement level keywords.
- **IETF RFC 2753**: Framework for Policy-based Admission Control (IETF, 2000). Establishes the Policy Decision Point (PDP), Policy Enforcement Point (PEP), and Policy Repository architectural pattern for network admission control. GiFo-RFC 0110 adapted this decomposition for AI governance; G-AGT extends it to operational governance engines. See §3.6.
- **IETF RFC 8032** (Ed25519): Signature algorithm for Type C adapter manifest attestation and AGT agent identity.
- **IETF RFC 8785** (JCS): JSON canonicalization for deterministic manifest signing. |
- **OASIS XACML 3.0**: eXtensible Access Control Markup Language (OASIS, 2003/2013). Defines the PEP/PDP/PAP/PIP decomposition for access control. GiFo-RFC 0110 formalized this decomposition for the AI governance domain; G-AGT acknowledges XACML as the earliest standardization of the P*P pattern. See §3.6.

- **W3C Verifiable Credentials Data Model 2.0**: Alternative credential representation for eIDAS 2.0 / EUDI wallet interoperability.
- **W3C Decentralized Identifiers** (DID) v1.0: Agent identity resolution. AGT references DID for agent identity. Published under the W3C Document License (royalty-free, freely implementable).

2. NOMENCLATURE

This section defines the unified terminology used throughout this specification, merging GAuth delegation governance terms with AGT-derived access control and operational governance terms.

2.1 Delegation Governance Terms (from GAuth)

Mandate: A structured Power of Attorney credential specifying the scope, constraints, and temporal bounds of an AI agent's authorization to act on behalf of a principal. | RFC 0111, RFC 0115.

PoA Credential: The machine-readable representation of a mandate, serialized as a JWT with PoA claims (RFC 0116 §4-5) or as a W3C Verifiable Credential (RFC 0116 §7). | RFC 0116.

Extended Token: A JWT carrying PoA attributes as claims, extending standard OAuth 2.1 access tokens with delegation-specific fields (governance_profile, allowed_verbs, budget, delegation_chain, etc.). | RFC 0116 §5.

PEP (Policy Enforcement Point): The component that intercepts agent action requests or calls and evaluates them against PoA credentials through a 16-check pipeline. Returns PERMIT, DENY, or CONSTRAIN. In G-AGT, the PEP is the authoritative governance control plane across all scenarios. The PEP defines a Phase 2 extension point (§4.2) for invoking external access control engines such as AGT. | RFC 0117.

PDP (Policy Decision Point): The component that evaluates mandate structure, governance profile ceilings, and policy rules to produce authorization decisions. | RFC 0110.

PAP (Policy Administration Point): The component through which policies and mandates are created, modified, and managed. Exposed via the Management API (RFC 0118). | RFC 0110.

PIP (Policy Information Point): The component that resolves contextual information needed for policy evaluation (agent identity, session state, budget status, etc.). | RFC 0110.

PVP (Policy Validation Point): The component that validates mandate structure and consistency against the PoA schema and governance profile ceilings. | RFC 0110.

Governance Profile: One of five predefined profiles (minimal, standard, strikt, enterprise, behoerde) controlling the strictness of agent governance, including approval mode, session limits, delegation depth, and budget ceilings. | RFC 0115 §4.

Three-Layer Capability Model: The structured permission model comprising Layer 1 (core action verbs), Layer 2 (permission-based capabilities), and Layer 3 (agent/platform-based capabilities). | RFC 0115 §3.

Delegation Chain: A sequence of delegations from principal to agent, potentially through intermediate agents, with scope narrowing at each step. | RFC 0115 §5.

Enforcement Decision: The output of PEP evaluation: PERMIT (all checks passed), DENY (one or more checks failed), or CONSTRAIN (permitted with restrictions). **CONSTRAIN** is produced exclusively by PEP Phase 1 — Phase 2 (AGT) does not produce or modify constraints. | RFC 0117 §5.

2.2 Adapter Type System

GAuth's connector model defines a 7-slot adapter framework with four adapter types:

```
| Type | Slots | License | Description |
|------|-------|---------|-------------|
| **Type A** | Slot 2: OAuthEngineAdapter | MPL 2.0 (user-replaceable) | OAuth engine adapter. The canonical example is Ory Hydra providing OIDC-based authorization as a vehicle within GAuth. G-AGT follows a similar vehicle integration pattern for AGT Engine, but via the PEP Phase 2 extension point (§4.2), not via a connector slot. |
| **Type B** | Slot 3: FoundryAdapter; Slot 4: WalletAdapter | MPL 2.0 (user-replaceable) | Action execution and secrets management. Foundry provides the agent marketplace within the Gimel platform (260+ agents). |
| **Type C** | Slot 5: GovernanceAdapter; Slot 6: Web3IdentityAdapter; Slot 7: DNAIdentityAdapter | Gimel Technologies ToS (sealed) | Proprietary adapters for excluded capabilities (AI governance, Web3, DNA/PQC). Require Ed25519 sealed manifest attestation. Available at Tariff M or higher. The open interface contracts are published under Apache 2.0 / MPL 2.0; only the implementations are proprietary. |
```

| **Type D** | (none assigned) | TBD | Future-reserved placeholder in the AdapterType union ("A" | "B" | "C" | "D"). No slot assigned, no interface defined. |

Important: The AGT Engine integration (Scenario i) does NOT occupy a connector slot. The PEP Phase 2 extension point (§4.2) is architecturally distinct from the Type A/B/C connector slot model. AGT is not subject to tariff gating, sealed manifest attestation, or connector slot registration. AGT's runtime lifecycle is owned by the AGT runtime itself.

2.3 AGT Terms

This specification distinguishes four uses of the term "AGT":

AGT: The Agent Governance Toolkit — a set of specifications for operational agent governance, e.g. in line with Microsoft's AGT codebase but not limited to it. Refers to the category of operational governance engines (also called “Platform-Bound Runtime Governance — PBRG”). Specifies broad operational governance capabilities: deterministic policy evaluation via YAML policy documents, execution privilege control (rings), deterministic trust scoring (5-tier), agent identity (Ed25519/DID/SPIFFE), kill switch, circuit breakers, SRE capabilities, tool connectors, memory management, and inter-agent communication. When deployed standalone (outside G-AGT), an AGT-compliant engine operates with its full feature set including its own audit logging and credential management.

AGT Engine: The G-AGT integration under Scenario i): AGT serves as an access control vehicle within GAuth's PEP Phase 2 extension point (§4.2). AGT provides only the vehicle function — the middleware hook (`PreToolUse`) that connects the agent runtime to GAuth's governance pipeline. GAuth absorbs and implements AGT's core access control primitives (deterministic policy evaluation, execution rings, trust scoring, kill switch, circuit breakers) natively within its own platform. AGT's native implementations of these capabilities are dormant (§4.4). Analogous to how the OAuth engine provides the OIDC protocol vehicle in the Type A adapter slot while GAuth orchestrates authorization around it.

AGT Bridge: The G-AGT integration under Scenario ii): AGT runs standalone with its complete feature set — all native capabilities active. AGT calls into GAuth's PEP for Phase 1 (credential-bound delegation enforcement) decisions via the Policy Translation Bridge (§6.2). AGT owns Phase 2 (platform-bound access control) natively, applying its own operational policies and governance controls. The bridge translates between GAuth's JSON and AGT's YAML worlds.

Microsoft AGT: Refers to the specific product, Microsoft`s Agent Governance Toolkit.

2.4 Access Control and Operational Governance

Access Control: The function performed by the AGT Engine integration in Scenario i). In this scenario, GAuth implements the access control capabilities natively (policy evaluation, ring enforcement, trust score checking, capability model validation); AGT provides only the vehicle function (middleware hook). The access control function is analogous to how the OAuth engine performs "authorization" in GAuth's Type A slot - a specific, bounded function within GAuth's architecture. | Scenario i) only.

Operational Governance: The broader function performed by AGT in Scenario ii): access control plus tool governance, memory management, agent marketplace integration, inter-agent communication security, and full SRE capabilities. All capabilities operate natively within AGT's runtime. AGT's native breadth — broader than access control alone. | Scenario ii) only.

2.5 Decision Vocabulary

GAuth and AGT use distinct decision vocabularies to avoid ambiguity:

```
| Component | Decision Values | Semantics |
|-----------|-----------------|-----------|
| **GAuth PEP (Phase 1)** | PERMIT, DENY, CONSTRAIN | Delegation authority decisions per RFC 0117. CONSTRAIN is produced exclusively by Phase 1. |
| **AGT Engine/Bridge (Phase 2)** | ALLOW, DENY | Policy evaluation decisions. Binary — no CONSTRAIN equivalent at the AGT level. Phase 2 cannot produce or modify constraints. Phase 2 can veto a Phase 1 CONSTRAIN (by returning DENY), but cannot alter the constraints attached to it. |
| **G-AGT Combined** | PERMIT, DENY, CONSTRAIN | Combined decision per §5.1.3. Constraint logic is 100% GAuth open core, unchanged by G-AGT integration. |
```

The PEP's CONSTRAIN decision carries mandate-imposed restrictions that are orthogonal to AGT's ALLOW/DENY. When the PEP returns CONSTRAIN and AGT returns ALLOW, the combined decision is CONSTRAIN — the action proceeds but subject to PEP-imposed constraints, unmodified by AGT.

2.6 Identity Terms

G-AGT operates with two complementary identity models that coexist within a single deployment:

Term	Definition	Layer
Human Identity	The identity established by GAuth through OAuth 2.1/OIDC tokens (JWT, W3C VC). This identity answers "who authorized this agent and on whose behalf does it act?" It is carried by the PoA credential and validated by PEP CHK-01 and CHK-02. This is the principal's identity — the human (or organizational entity) that granted the mandate.	GAuth (delegation governance)
Agent Identity	The identity established by AGT through cryptographic mechanisms native to the operational governance engine (e.g., Ed25519 keys, W3C DID, SPIFFE/SVID certificates in AGT's AgentMesh). This identity answers "is this agent runtime instance cryptographically authentic?" It is validated by AGT's zero-trust identity layer.	AGT

Both identity layers Must be present and valid for an agent to operate in a G-AGT deployment. They are complementary, not competing: human identity establishes authorization authority; agent identity establishes runtime authenticity.

2.7 Scoring Domains

G-AGT maintains strict separation between two scoring domains:

Term	Definition	Domain	Scale	Method	Purpose
Trust Score	A deterministic, rule-based score reflecting an agent's mid-term behavioural reliability. Computed from compliance counters, violation history, identity verification strength, delegation chain depth, budget utilization, and session compliance. Enables execution ring assignment and governance profile enforcement.	AGT (operational)	0–1000 integer, 5-tier (§4.3.3)	Deterministic only. No AI/ML.	Ring assignment, behavioral gating
Confidence Score	A probabilistic score produced by the proprietary GovernanceAdapter (Slot 5, Type C) reflecting the AI-assessed likelihood that an action falls within implied				

```
authority. Used exclusively for implied authority override
when deterministic evaluation produces DENY. | GAuth
(proprietary, Type C Slot 5) | 0.0–1.0 float | AI/ML-enhanced.
Proprietary. | Implied authority override |
```

Both scores are preserved independently in the unified audit record (§9). The trust score Must Not be overwritten, replaced, or blended with the confidence score. They are architecturally distinct signals serving different governance functions.

The Confidence Score is not in scope of this RFC0140 and its license (Exclusion).

2.8 G-AGT Integration Terms

G-AGT: The integration profile defined by this specification (GiFo-RFC 0140), integrating AGT-based operational governance engines within GAuth's authorization architecture. Covers Scenario i) (AGT Engine), Scenario ii) (AGT Bridge), and the Dynamic Scenario.

Phase 1 - Credential-Bound Delegation Enforcement: The evaluation phase performed by GAuth's PEP (16-check pipeline, RFC 0117 §9). Evaluates whether the specific authority granted via the agent's credential (mandate/PoA) permits the requested action. The compliance rules enforced in Phase 1 - sectors, regions, verbs, budget, session limits, governance profile constraints, delegation chain scope - are bound to the individual credential. Phase 1 does not merely validate that a credential exists; it enforces the full organizational compliance posture encoded within that credential. See §5.1.

Phase 2 — Platform-Bound Access Control: The evaluation phase performed by GAuth natively (Scenario i) or by the AGT engine (Scenario ii). Evaluates the action against platform-level operational policies configured independently of any specific credential — cross-cutting rules (YAML, JSON, Cedar, OPA/Rego policy documents) that apply to all agents regardless of their individual mandates. See §5.1.

PEP Phase 2 Extension Point: The architectural mechanism by which GAuth's PEP invokes an external access control engine (such as AGT) for Phase 2 (platform-bound access control) evaluation after completing Phase 1 (credential-bound delegation enforcement). This is NOT a connector slot — it is not subject to tariff gating, sealed manifest attestation, or connector slot registration. The AGT runtime owns its own lifecycle. See §4.2.

Policy Translation Bridge: The component used in Scenario ii) only that translates between GAuth's structured JSON world (PoA credentials, enforcement requests, JWT claims) and AGT's native YAML-based policy evaluation language. The bridge is an additional policy language adapter. Not needed in Scenario i), where GAuth and AGT share a native JSON context. See §6.

Mandate-Aware Policy Rule: An AGT policy rule that references GAuth PoA claim fields in its conditions, enabling access control or operational policy decisions that depend on delegation context (e.g., governance profile, remaining budget, delegation depth).

Unified Audit Record: A merged audit log entry where GAuth's enforcement audit is authoritative and AGT operational telemetry is included as supplementary data, providing a single compliance-ready record of every evaluated agent action. Preserves both trust score and confidence score independently.

Budget Deferral: The transactional semantics pattern (§5.2) by which the stateful PEP places a tentative budget hold during Phase 1 evaluation, then commits the hold on Phase 2 ALLOW or releases it on Phase 2 DENY. Prevents budget consumption for actions subsequently denied by AGT.

Gimel-AGT: A deployment of G-AGT that additionally incorporates proprietary Gimel services via Type C adapters. Gimel-AGT is a commercial product built on the open G-AGT specification. It is not defined by this RFC.

Dormant Feature: An AGT-native feature (e.g., audit logging, credential management) that is functionally redundant with a GAuth-native capability and therefore inactive in a G-AGT deployment. The feature remains technically present in AGT for standalone deployments outside the GAuth ecosystem.

3. WHY G-AGT

3.1 The Governance Gap

AI agents operating autonomously - making decisions, entering transactions, invoking tools, and communicating with other agents - require governance at two distinct levels:

(a) **Delegation governance** answers: "Has this agent been authorized to perform this action on behalf of a principal? What is the scope of that authorization? Is the authorization still valid, within budget, and not revoked?"

(b) **Access control / operational governance** answers: "Is this agent permitted to invoke this specific tool in this execution environment? What privilege level does it operate at? Is it behaving within its deterministic trust parameters? Are its operational SLOs being met?"

Neither paradigm alone provides complete AI governance:

- Delegation governance without access control allows an agent with a valid mandate to invoke dangerous tools, execute in an unsandboxed environment, or cascade failures across multi-agent systems — all within the scope of its authorization.

- Access control without delegation governance allows an agent to act without verifiable authorization from a principal, with no structured accountability chain, no transparent scope limitations, and no ability for relying parties to verify the agent's authority.

3.2 Deep Authority vs. Broad Runtime

GAuth (GiFo-RFCs 0110 - 0118) governs authority deeply: verifiable PoA credentials, mandate lifecycle management, a 16-check enforcement pipeline, governance profiles, budget controls, delegation chains, and tariff-gated adapters. It answers the "who authorized this agent and what are they allowed to decide/do" question comprehensively.

AGT governs everything broadly with a uniform policy engine: tool invocation policies, execution sandboxing through privilege rings, deterministic trust scoring (5-tier), SRE capabilities (SLOs, circuit breakers), agent lifecycle management, memory management, inter-agent communication, and agent marketplace integration. It answers the "is this agent operationally safe to execute this action right now" question.

G-AGT positions GAuth as the authority and policy layer that AGT's runtime hooks call into - not a system that tries to replicate AGT's breadth. The integration follows the same vehicle pattern that GAuth uses for OAuth engines:

- Just as Ory Hydra serves as the OAuth vehicle in GAuth's Type A adapter slot (providing OIDC-based authorization as a vehicle for GAuth's delegation model), AGT serves as the access control vehicle (providing the middleware hook as a vehicle for GAuth's PEP enforcement) in Scenario i). In Scenario i), GAuth implements the access control capabilities natively; AGT provides only the runtime connection point.
- In Scenario ii), AGT retains its full breadth and all native capabilities, calling into GAuth for Phase 1 (credential-bound delegation enforcement) decisions only. AGT owns Phase 2 (platform-bound access control) natively.
- In both scenarios, GAuth remains the authoritative governance control plane. AGT does not make independent governance authority decisions - it either provides the vehicle function at the direction of the GAuth PEP (Scenario i), or calls the PEP for authority decisions before permitting actions (Scenario ii).

3.3 Integration Scenario Spectrum

The three scenarios form a spectrum of integration depth:

```
| Scenario | AGT Role | AGT Provides | GAuth Provides | When to Use |
```

```
|----------|----------|-------------|----------------|--------
-----|

| **i) AGT Engine** | Access control vehicle | Vehicle
function only (middleware hook). Native capabilities dormant.
| Phase 1 (PEP 16-check) + Phase 2 (policy eval, rings, trust
scoring, kill switch, circuit breakers) — all implemented
natively by GAuth. | Clean architecture, consistent platform,
new deployments |

| **ii) AGT Bridge** | Full operational governance engine |
All capabilities natively (policy eval, rings, trust scoring,
kill switch, circuit breakers, tool governance, memory,
marketplace, comms) | Phase 1 only (PEP 16-check via
translation bridge) | Existing AGT deployments, need AGT's
broader connector ecosystem |

| **Dynamic** | Full GAuth implementation | Full GAuth spec
suite (RFCs 0115-0118) | G-AGT provides Type C gateway +
conformance authority | Future evolution |
```

3.4 Foundry Availability

The Foundry Type B connector (Slot 3: FoundryAdapter) - providing the Gimel platform's agent marketplace with 260+ agents - is available in all G-AGT integration scenarios. In Scenario i) (AGT Engine), the Foundry connector is accessed through GAuth's adapter framework. In Scenario ii) (AGT Bridge), the Foundry connector is accessible through both GAuth's adapter framework and AGT's native connector ecosystem. In the Dynamic Scenario, the Foundry connector remains available as a Type B adapter under MPL 2.0.

3.5 Redundancy Management (Dormancy)

Standalone AGT provides capabilities that overlap with GAuth's native features. In a G-AGT deployment (Scenario i or ii), the following dormancy rules apply:

```
| Capability | GAuth Native | AGT Native | G-AGT Behavior
(Scenario i) | G-AGT Behavior (Scenario ii) |

|-----------|-------------|-----------|----------------------
-----|-----------------------------|

| Audit logging | Enforcement audit (RFC 0117 §5.1) —
authoritative, compliance-ready, per-check granularity |
Flight recorder — operational telemetry | GAuth audit
authoritative. AGT flight recorder dormant; telemetry merged
```

as supplementary. | GAuth audit authoritative. AGT flight recorder dormant; telemetry merged as supplementary. |

| Credential management | PoA credential lifecycle (RFC 0118), JWT/W3C VC issuance, mandate status management | Agent registration, credential rotation | GAuth authoritative. AGT credential management dormant. | GAuth authoritative. AGT credential management dormant. |

| Identity | Human identity via OAuth 2.1/OIDC (JWT, W3C VC) | Agent identity via Ed25519/DID/SPIFFE | Both active — complementary (§2.6). | Both active — complementary (§2.6). |

| Policy evaluation | PEP 16-check pipeline (credential-bound delegation enforcement) | Policy Evaluator (platform-bound policy rules) | GAuth implements Phase 2 (platform-bound access control) natively. AGT policy evaluator dormant. | Both active — complementary. PEP evaluates credential-bound delegation (Phase 1); AGT evaluates platform-bound operational policy (Phase 2) natively. |

| Execution rings | (implemented natively by GAuth in Scenario i) | 4-tier ring model | GAuth implements rings natively. AGT ring model dormant. | AGT rings active natively. |

| Trust scoring | (implemented natively by GAuth in Scenario i) | 5-tier deterministic trust scoring | GAuth implements trust scoring natively (same 5-tier model). AGT trust scoring dormant. | AGT trust scoring active natively. |

| Kill switch | (implemented natively by GAuth in Scenario i) | Immediate agent termination | GAuth implements kill switch natively. AGT kill switch dormant. | AGT kill switch active natively. |

| Circuit breakers | (implemented natively by GAuth in Scenario i) | Cascading failure prevention | GAuth implements circuit breakers natively. AGT circuit breakers dormant. | AGT circuit breakers active natively. |

| Budget management | Budget operations (RFC 0118 §7) — ceiling, consumption, exhaustion | (not typically provided) | GAuth only. Budget deferral semantics (§5.2). | GAuth only. |

| Delegation chain | CHK-16 delegation chain enforcement (RFC 0117) | (not typically provided) | GAuth only. | GAuth only. |

AGT features marked as "dormant" remain technically functional within the AGT engine. They are not disabled or removed — they are simply not consulted by the G-AGT

integration. This ensures AGT can operate independently in standalone deployments outside the GAuth ecosystem.

3.6 Architectural Prior Art

The P*P (Power*Point or Policy *Point, respectively) architectural decomposition - separating governance into Policy Enforcement Point (PEP), Policy Decision Point (PDP), Policy Administration Point (PAP), and Policy Information Point (PIP) - has a well-established lineage in access control and network policy standards:

Prior Art Chain:

IETF RFC 2753: (Framework for Policy-based Admission Control) | 2000 | Established the PDP/PEP/Policy Repository decomposition for network admission control. First IETF formalization of separating policy decisions from policy enforcement. | Foundational architectural pattern. G-AGT's PEP/PDP separation traces to this framework.

OASIS XACML 1.0/3.0: (eXtensible Access Control Markup Language) | 2003/2013 | Extended RFC 2753's decomposition into a full P*P model (PEP, PDP, PAP, PIP) for access control. Introduced standardized policy language, obligation expressions, and multi-valued decisions (Permit/Deny/Indeterminate/Not Applicable). | Direct architectural ancestor. GAuth's PEP 16-check pipeline and PERMIT/DENY/CONSTRAIN vocabulary are informed by XACML's decision model.

GiFo-RFC 0110: (GAuth Protocol Engine) | 2026 | First application of the P*P decomposition to the AI agent governance domain. Added PVP (Policy Validation Point) for credential schema validation. Established the pattern of governance-over-delegation that distinguishes AI agent authorization from traditional access control. | GAuth's native P*P architecture. G-AGT extends this with Phase 2 (AGT) as a complementary governance layer.

G-AGT (this specification): 2026 | Extends GiFo-RFC 0110's AI governance P*P model with an operational governance engine (AGT) providing Phase 2 access control. Defines the PEP Phase 2 extension point as the integration mechanism. | Current specification.

AGT's policy evaluation architecture follows the same P*P decomposition:

- AGT's Policy Evaluator corresponds to the PDP (Policy Decision Point) — evaluating policy rules against action requests to produce access control or operational governance decisions.
- AGT's YAML policy document management corresponds to the PAP (Policy Administration Point) — administering the policy rules that govern agent behavior.
- AGT's PreToolUse enforcement hook corresponds to the PEP (Policy Enforcement Point) — intercepting agent actions and enforcing policy decisions at the point of execution.

- AGT's context resolution layer (agent identity, trust score, execution ring status) corresponds to the PIP (Policy Information Point) — resolving contextual information needed for policy evaluation.

GiFo-RFC 0110 thus serves as the first application of the P*P pattern to the AI governance domain. While AGT extends the pattern with domain-specific capabilities (execution rings, deterministic trust scoring, circuit breakers, SRE), the foundational decomposition of governance into enforcement, decision, administration, and information components was formalized by IETF RFC 2753, standardized by OASIS XACML, and adapted for AI governance by RFC 0110.

3.7 Standards-Based Interoperability

G-AGT is designed as an interoperability specification, not a product specification. Any operational governance engine that implements the AGT access control interfaces defined in §4 can serve as an engine within GAuth's PEP Phase 2 extension point (Scenario i) or connect via the translation bridge (Scenario ii). Similarly, any GAuth-compliant authorization server (per RFCs 0110-0118) can orchestrate an AGT engine. This ensures vendor independence and future-proofing without creating competing governance authorities.

3.8 Engine Pattern Extensibility

The P*P architecture formalized by GiFo-RFC 0110 - and rooted in IETF RFC 2753 and OASIS XACML - is engine-agnostic. While this specification focuses on AGT as the exemplar operational governance engine, the same architectural pattern accommodates additional engine types that address distinct governance domains. This section identifies three anticipated engine patterns that follow the P*P decomposition and could integrate with GAuth's architecture using the same PEP Phase 2 extension point defined herein. Full specifications for these patterns are deferred to future GiFo-RFCs.

3.8.1 Policy-as-Code Engines

Organizations with established policy-as-code infrastructure — such as Open Policy Agent (OPA/Rego), Cedar (AWS), or Zanzibar/SpiceDB (Google) — may wish to use these engines as the access control vehicle within GAuth's PEP Phase 2 extension point rather than adopting AGT's YAML-based policy language. The P*P decomposition applies directly:

```
| P*P Component | AGT (this specification) | OPA/Rego | Cedar
| Zanzibar/SpiceDB |

|--------------|-------------------------|----------|-------|-
---------------|

| **PDP** (Policy Decision) | Policy Evaluator (YAML rules) |
OPA evaluation engine (Rego queries) | Cedar authorization
engine (Cedar policy language) | SpiceDB check engine
(relationship tuples) |

| **PAP** (Policy Administration) | YAML policy document
management | OPA bundle server / policy store | Cedar policy
store | Schema and relationship management |

| **PEP** (Policy Enforcement) | PreToolUse hook |
Application-embedded enforcement | Application-embedded
enforcement | Application-embedded enforcement |

| **PIP** (Policy Information) | Agent identity, trust score,
ring status | External data sources, JWT claims | Entity
attributes, context | Relationship graph |
```

Any policy-as-code engine that satisfies the access control engine requirements defined in §4.3 - deterministic policy evaluation, execution privilege control, deterministic trust scoring (5-tier), capability model enforcement, and kill switch - can serve in the AGT Engine role via the PEP Phase 2 extension point (Scenario i) or connect via a translation bridge (Scenario ii). The policy language changes; the integration architecture and GAuth's authoritative role remain the same.

This extensibility is inherent in G-AGT's design: the §4.3 requirements are specified in terms of capabilities and behaviours, not implementation technology. An OPA/Rego engine that provides deterministic evaluation, four execution rings, trust scoring on the 0–1000 scale with 5-tier boundaries and kill switch capability is a valid G-AGT Core conformant engine (§10.2).

3.8.2 Agent Communication Governance

As agent-to-agent protocols mature — including the Model Context Protocol (MCP), Google's Agent-to-Agent (A2A) protocol, and emerging multi-agent coordination standards — there is an anticipated need for a communication governance engine: a governance layer that addresses not what an agent does (AGT's domain) or who authorized it (OAuth/GAuth's domain), but how agents communicate, coordinate, and federate.

This third governance domain would address:

- **Message filtering and routing policies** — deterministic rules governing which agents may communicate, through which channels, and with what content restrictions.
- **Protocol compliance** — enforcement of agent-to-agent protocol standards (message format, capability negotiation, session lifecycle).
- **Multi-agent orchestration governance** — policies governing agent coordination patterns (fan-out limits, delegation depth in multi-agent chains, resource allocation across agent pools).
- **Cross-organizational agent federation** — governance rules for agents operating across organizational boundaries, including trust establishment, credential exchange, and jurisdictional compliance.

The P*P decomposition applies to this domain: message policy evaluation (PDP), communication policy administration (PAP), message interception and enforcement (PEP), and agent registry / federation context resolution (PIP). GAuth would remain the authoritative governance control plane, providing the delegation authority and mandate scope within which communication governance operates.

A full specification for agent communication governance is anticipated for future GiFo-RFC coverage. The architectural foundation — RFC 0110's P*P pattern (rooted in IETF RFC 2753 and OASIS XACML), GAuth's PEP Phase 2 extension point, and the integration patterns defined in this specification — provides the structural basis for this extension.

3.8.3 Compliance and Regulatory Engines

A deterministic compliance engine that maps agent actions to regulatory requirements — EU AI Act, NIST AI RMF, sector-specific regulations (financial services, healthcare, public administration) — represents another valid engine pattern within the P*P framework. The P*P decomposition applies:

- **PDP** (Policy Decision): Regulatory rule evaluation — matching agent actions against deterministic compliance checklists, regulatory requirement mappings, and jurisdictional obligation tables.
- **PAP** (Policy Administration): Regulation database management — maintaining structured representations of regulatory requirements, compliance rule sets, and jurisdictional mappings.
- **PEP** (Policy Enforcement): Action interception for compliance checks — intercepting agent actions at the point of execution to evaluate regulatory compliance before permitting the action.

- **PIP** (Policy Information): Jurisdictional and contextual information resolution — determining applicable regulatory frameworks based on agent location, data residency, sector classification, and principal jurisdiction.

Open-source scope: Deterministic, rule-based compliance governance — regulatory checklists, jurisdiction mapping tables, deterministic compliance scoring, and rule-based regulatory requirement matching — is a valid open-source engine pattern within the P*P framework, subject to the same Apache 2.0 license as this specification. Anyone may build a deterministic compliance engine that integrates with GAuth's PEP Phase 2 extension point using the patterns defined herein.

Proprietary boundary: Effective compliance governance at scale typically requires capabilities beyond deterministic rule matching: adaptive risk assessment that learns from emerging regulatory interpretations, learning-loop regulatory monitoring that tracks evolving compliance landscapes, stochastic compliance analysis for probabilistic risk quantification, and AI-powered regulatory interpretation that maps natural-language regulations to machine-enforceable policies. These AI-enhanced compliance capabilities remain exclusively available through the proprietary GovernanceAdapter (Slot 5, §8.2), consistent with the exclusions defined in §1.1. The boundary is clear: deterministic compliance is open; AI-enhanced compliance is proprietary via Type C.

A full specification for compliance engine integration is anticipated for future GiFo-RFC coverage.

4. WHAT G-AGT IS

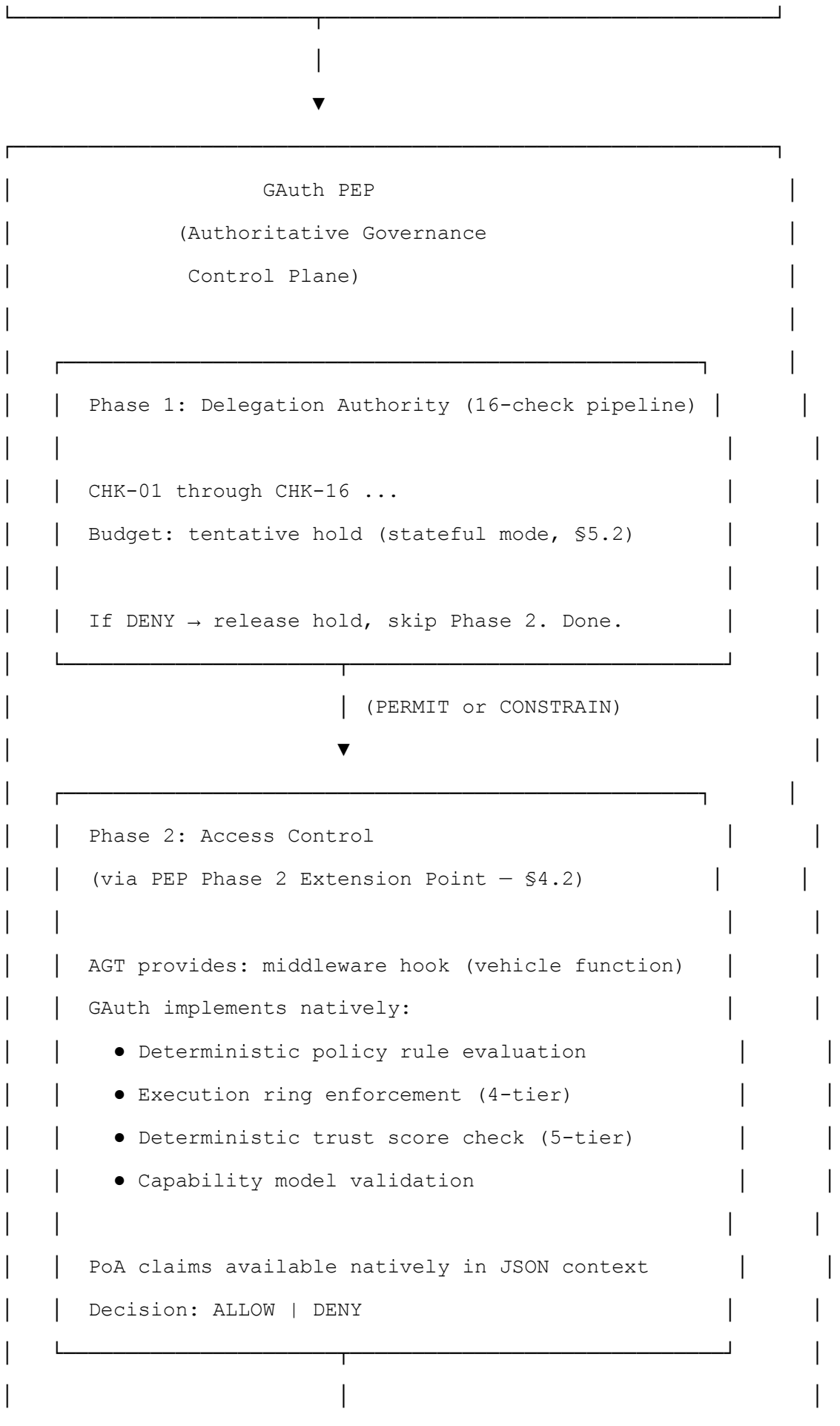
4.1 Architecture Overview

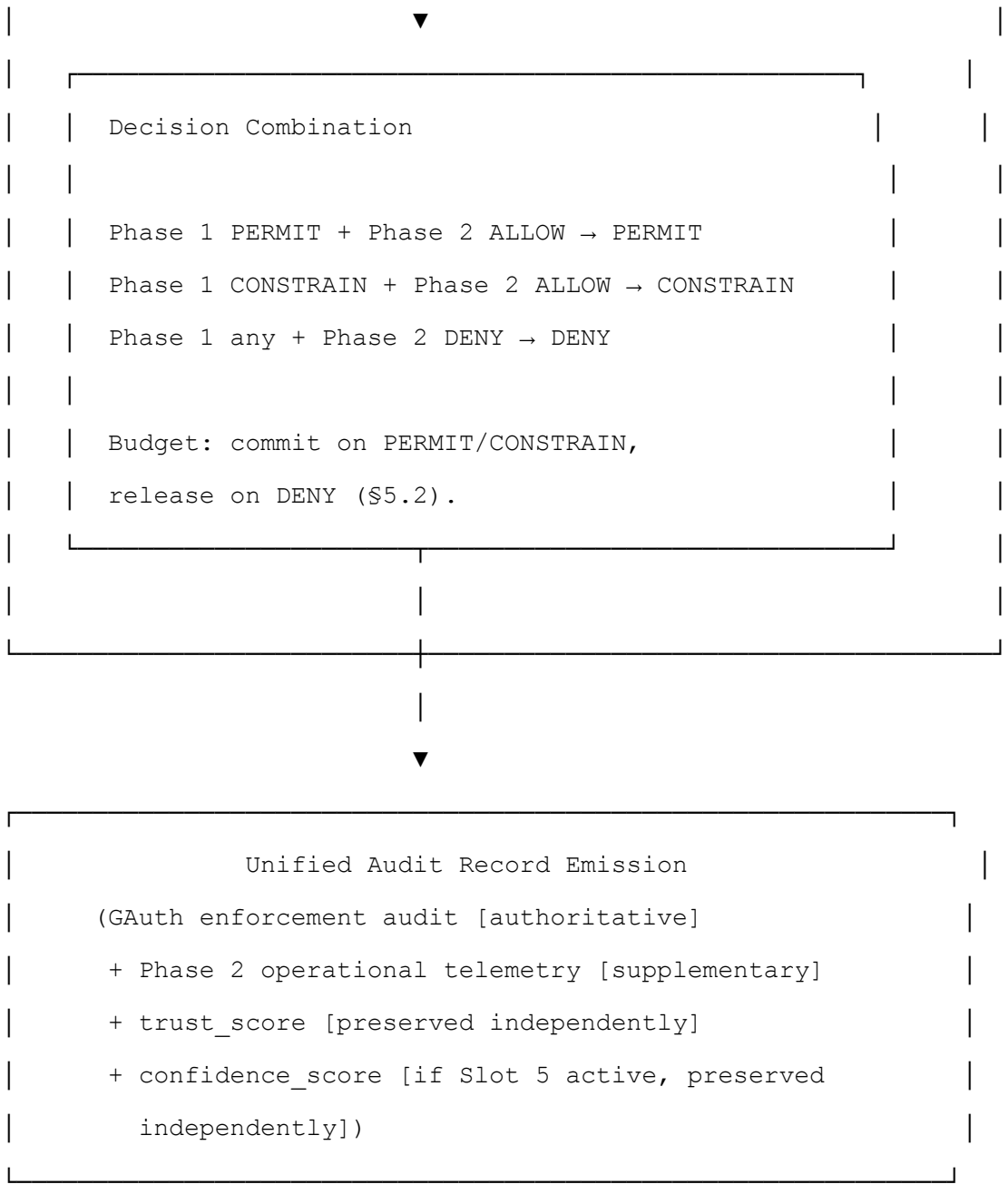
G-AGT integrates AGT within GAuth's P*P architecture (RFC 0110). The architecture differs by scenario:

4.1.1 Scenario i) — AGT Engine Architecture

GAuth's PEP is the authoritative governance control plane. AGT provides only the vehicle function (middleware hook). GAuth implements core access control capabilities natively. AGT's runtime hooks (e.g., `PreToolUse`) call the PEP:

```
┌──────────────────────────────────────────────────────────────┐
│                    Agent Action Request                      │
│         (verb, resource, parameters, agent context)          │
```


GAuth PEP
(Authoritative Governance
Control Plane)
Phase 1: Delegation Authority (16-check pipeline)
CHK-01 through CHK-16 ...
Budget: tentative hold (stateful mode, §5.2)
If DENY → release hold, skip Phase 2. Done.
(PERMIT or CONSTRAIN)
Phase 2: Access Control
(via PEP Phase 2 Extension Point — §4.2)
AGT provides: middleware hook (vehicle function)
GAuth implements natively:
• Deterministic policy rule evaluation
• Execution ring enforcement (4-tier)
• Deterministic trust score check (5-tier)
• Capability model validation
PoA claims available natively in JSON context
Decision: ALLOW | DENY

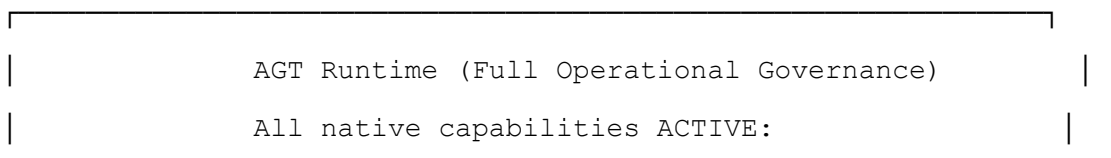


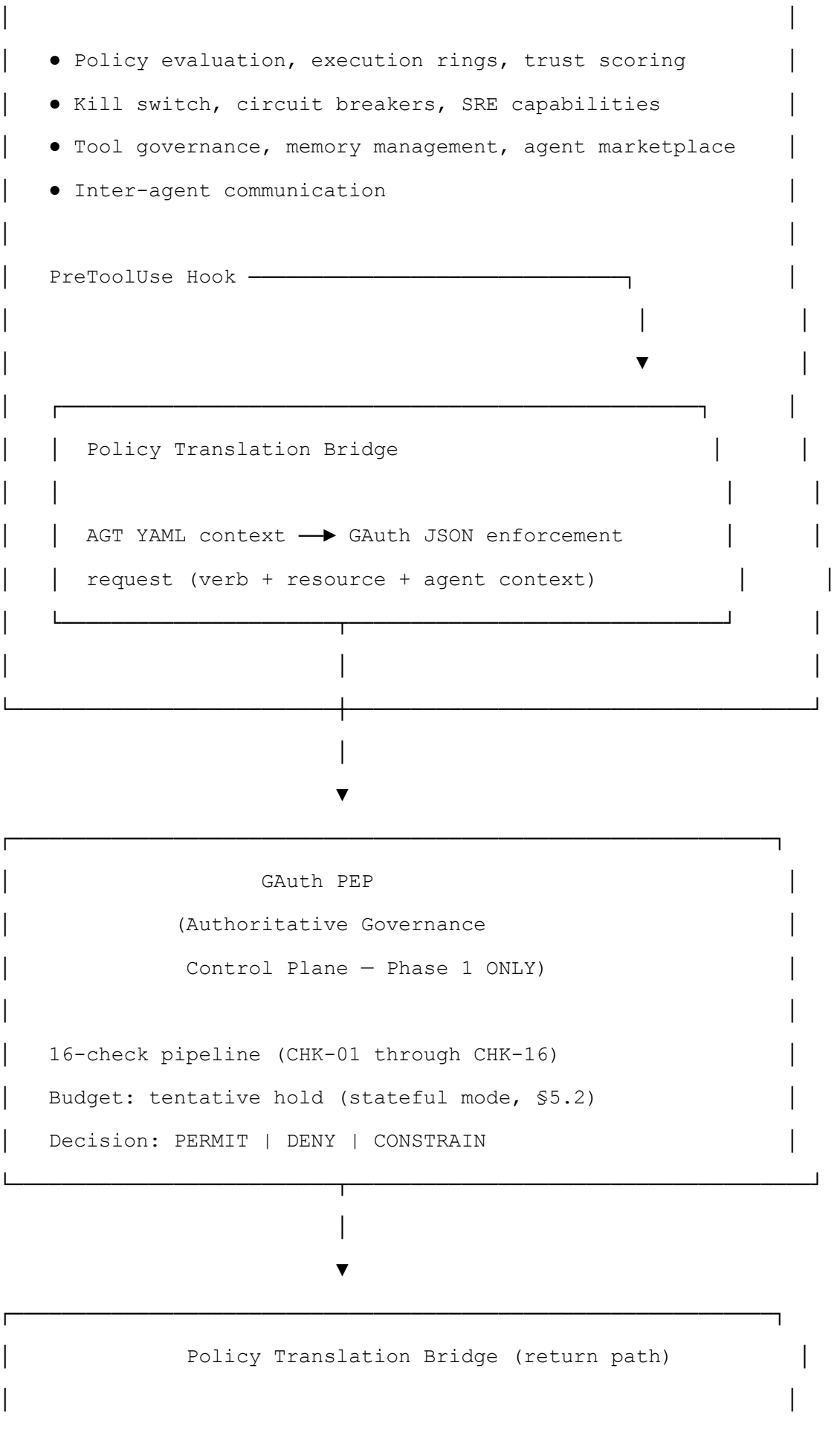

```
```

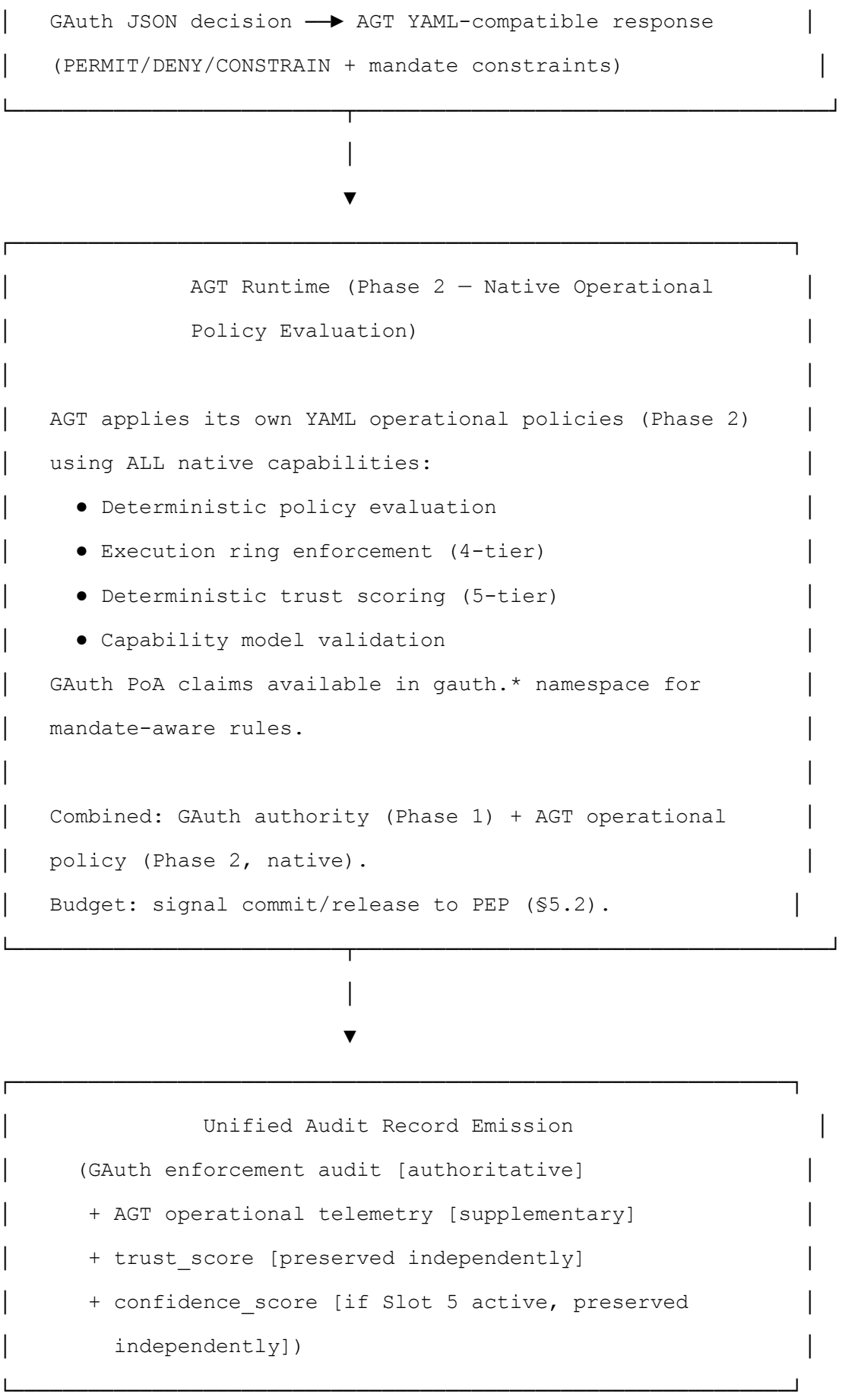
4.1.2 Scenario ii) — AGT Bridge Architecture

AGT runs standalone with its complete feature set. AGT calls into GAuth's PEP for Phase 1 (credential-bound delegation enforcement) decisions. AGT owns Phase 2 (platform-bound access control) natively:

```
```




• Policy evaluation, execution rings, trust scoring
• Kill switch, circuit breakers, SRE capabilities
• Tool governance, memory management, agent marketplace
• Inter-agent communication
PreToolUse Hook
Policy Translation Bridge
AGT YAML context ⟶ GAuth JSON enforcement
request (verb + resource + agent context)
GAuth PEP
(Authoritative Governance
Control Plane — Phase 1 ONLY)
16-check pipeline (CHK-01 through CHK-16)
Budget: tentative hold (stateful mode, §5.2)
Decision: PERMIT | DENY | CONSTRAIN
Policy Translation Bridge (return path)


GAuth JSON decision ⟶ AGT YAML-compatible response
(PERMIT/DENY/CONSTRAIN + mandate constraints)
AGT Runtime (Phase 2 — Native Operational Policy Evaluation)
AGT applies its own YAML operational policies (Phase 2) using ALL native capabilities:
• Deterministic policy evaluation
• Execution ring enforcement (4-tier)
• Deterministic trust scoring (5-tier)
• Capability model validation
GAuth PoA claims available in gauth.* namespace for mandate-aware rules.
Combined: GAuth authority (Phase 1) + AGT operational policy (Phase 2, native).
Budget: signal commit/release to PEP (§5.2).
Unified Audit Record Emission
(GAuth enforcement audit [authoritative]
+ AGT operational telemetry [supplementary]
+ trust_score [preserved independently]
+ confidence_score [if Slot 5 active, preserved independently])

```
```

4.2 The PEP Phase 2 Extension Point

The AGT Engine integration (Scenario i) uses a PEP Phase 2 extension point — an architectural mechanism that is distinct from GAuth's connector slot model (Type A/B/C).

4.2.1 Extension Point vs. Connector Slot

Aspect	Connector Slot (Type A/B/C)	PEP Phase 2 Extension Point
Registration	ConnectorSlotRegistry, explicit slot assignment	PEP configuration, no slot assignment
Tariff gating	Yes — Type C requires Tariff M+	No — not subject to tariff gating
Sealed manifest	Type C requires Ed25519 attestation	No attestation required
Lifecycle ownership	GAuth manages adapter lifecycle (null → pending → active → error)	AGT runtime manages its own lifecycle
Replaceability	Per adapter type rules (Type A user-replaceable, Type C sealed)	Any G-AGT Core conformant engine (§10.2)
Invocation	Adapter interface methods via ConnectorSlotRegistry	PEP invokes Phase 2 engine after Phase 1 completion
Failure behavior	Fail-closed per adapter type	Fail-closed (DENY if engine unreachable)

4.2.2 Extension Point Contract

The PEP Phase 2 extension point defines a minimal contract:

- **Input**: The PEP provides the Phase 2 engine with: (a) the Phase 1 decision (PERMIT or CONSTRAIN — DENY short-circuits before Phase 2), (b) the full `gauth` namespace (§6.1) containing PoA claims, governance profile, budget

status, and PEP evaluation metadata, and (c) the original action request (verb, resource, agent context).

- **Output**: The Phase 2 engine returns a binary decision (ALLOW or DENY) with a reason string and optional metadata (rules evaluated, trust score, execution ring, processing time).
- **Lifecycle**: The AGT runtime owns the Phase 2 engine's lifecycle. GAuth does not manage AGT process startup, shutdown, health monitoring, or version upgrades. GAuth's responsibility is limited to: invoking the engine, handling its response, and failing closed if the engine is unreachable.

4.2.3 Rationale

The PEP Phase 2 extension point is not a connector slot because:

1. **No tariff dependency.** Access control is a core governance function, not a premium feature. Gating Phase 2 behind tariff tiers would undermine the defense-in-depth architecture.

2. **No sealed manifest.** AGT is an open-source engine (MIT license). Requiring Ed25519 attestation for an open engine contradicts the open integration philosophy.

3. **Different lifecycle model.** Connector slots have GAuth-managed lifecycles (null → pending → active → error). AGT runtimes manage their own lifecycle — GAuth should not assume control over AGT process management.

4. **Architectural consistency.** The PEP already defines the evaluation pipeline (Phase 1). The Phase 2 extension point is a natural extension of the PEP's evaluation model, not an adapter plugged into a registry.

4.2.4 Scenario ii) Symmetry

In Scenario ii), the extension point pattern is inverted: AGT's `PreToolUse` hook acts as the extension point that calls GAuth's PEP. The same contract applies in reverse — AGT provides action context to the PEP, receives a Phase 1 decision, and combines it with its own Phase 2 evaluation.

4.3 Phase 2 Access Control Requirements

The following requirements define the capabilities that Must be present in the Phase 2 access control layer of any G-AGT deployment. Who implements them differs by scenario:

- **In Scenario i),** GAuth implements these capabilities natively within its own platform. The AGT engine's native implementations of these capabilities are dormant (§4.4.1). The requirements below specify what GAuth Must implement.
- **In Scenario ii),** the AGT engine implements these capabilities natively. The requirements below specify what the AGT engine Must provide.

In both scenarios, the functional requirements are identical — only the implementing party differs:

4.3.1 Deterministic Policy Evaluation

The Phase 2 implementation (GAuth-native in Scenario i, AGT-native in Scenario ii) Must provide a policy evaluator that:

- Accepts structured action requests containing at minimum: action verb (in the GAuth verb taxonomy format `urn:gauth:verb:{domain}:{category}:{action}` per §6.5), target resource, requesting agent identifier, and evaluation context (including GAuth PoA claims — natively in Scenario i per §6.1, or via translation in Scenario ii per §6.2).
- Evaluates action requests against a set of policy rules deterministically — the same input Must always produce the same output. No AI, ML, probabilistic, or stochastic methods are permitted in the evaluation path.
- Returns a binary decision (ALLOW or DENY) with a human-readable reason and, optionally, a list of violated policy rules.
- Supports policy rules expressible as condition-effect pairs, where conditions reference fields from the action request and evaluation context (including `gauth.*` namespace fields).

4.3.2 Execution Privilege Control

The Phase 2 implementation Must support at least four execution privilege levels (rings), ordered from highest privilege (Ring 0) to most restricted (Ring 3):

```
| Ring | Privilege Level | Description |
|------|----------------|-------------|
| Ring 0 | Maximum privilege | Full tool access, minimal isolation. Reserved for highly trusted agents (trust tier: Trusted or Verified Partner) with restrictive mandates. |
| Ring 1 | Standard privilege | Standard tool access with monitoring. Default for agents with active mandates under
```

```
standard or strikt governance profiles (trust tier: Standard
or above). |

| Ring 2 | Restricted privilege | Limited tool access,
enhanced monitoring, mandatory approval gates for sensitive
operations. Default for enterprise/behoerde profiles or agents
with Probationary trust tier. |

| Ring 3 | Minimum privilege | Read-only access, maximum
isolation, all actions require explicit approval. Assigned to
agents with Untrusted trust tier. |
```

The Phase 2 implementation Must allow ring assignment based on evaluation context, including GAuth governance profile claims and the 5-tier trust score (§4.3.3).

4.3.3 Deterministic Trust Scoring

The Phase 2 implementation Must maintain a trust score for each registered agent on a normalized integer scale of 0 to 1000, with five named trust tiers:

```
| Range | Trust Tier | Typical Ring Assignment | Typical
Treatment |

|-------|-----------|------------------------|----------------
---|

| 0 – 299 | **Untrusted** | Ring 3 | Agent blocked or requires
manual approval for all actions. Maximum isolation. |

| 300 – 499 | **Probationary** | Ring 2 | Restricted tool
access. Enhanced monitoring. Mandatory approval gates for
sensitive operations. |

| 500 – 699 | **Standard** | Ring 1 | Standard tool access.
Normal operational monitoring. Default tier for newly
registered agents with verified identity. |

| 700 – 899 | **Trusted** | Ring 1 or Ring 0 | Elevated
access. Reduced monitoring frequency. Ring 0 eligible with
restrictive mandate. |

| 900 – 1000 | **Verified Partner** | Ring 0 | Maximum
privilege. Reserved for agents with extensive compliance
history and verified organizational identity. |
```

Trust scores Must be computed exclusively from deterministic, rule-based signals. The following computation inputs are permitted:

- **Compliance counters** — Number of policy-compliant actions vs. violations, computed as simple ratios or weighted sums with fixed coefficients.

- **Violation history** — Count and severity of past policy violations, with configurable decay windows (e.g., violations older than 30 days carry reduced weight).
- **Identity verification strength** — Categorical scoring based on identity verification method (e.g., Ed25519-verified = +200, DID-resolved = +175, SPIFFE/SVID = +150, unverified = 0).
- **Delegation chain depth** — Fixed penalty per delegation level (e.g., -50 per level of delegation depth).
- **Budget utilization thresholds** — Fixed score adjustments at configurable utilization thresholds (e.g., >90% utilization = -100).
- **Session compliance** — Ratio of approved vs. denied actions in the current session.

The following computation methods are explicitly prohibited in the open G-AGT specification:

- Machine learning models (supervised, unsupervised, reinforcement learning).
- Neural networks or deep learning of any kind.
- Adaptive or self-modifying scoring algorithms that change their behavior based on accumulated data.
- Probabilistic or stochastic scoring methods.
- Natural language processing or semantic analysis of agent behavior.
- Any method that produces non-deterministic outputs for identical inputs.

Trust Score vs. Confidence Score: The deterministic trust score defined in this section is architecturally distinct from the confidence score produced by the proprietary GovernanceAdapter (Slot 5, §8.2). The trust score is an AGT-domain signal (0–1000 integer, deterministic, 5-tier) that enables execution ring assignment and behavioural gating. The confidence score is a GAuth-domain signal (0.0–1.0 float, AI-enhanced, proprietary) that supports implied authority override. Both scores are preserved independently in the unified audit record (§9) — the confidence score Must Not overwrite, replace, or blend with the trust score. When the GovernanceAdapter (Slot 5) is active, it May produce a confidence score alongside the deterministic trust score, but the two scores serve different functions and Must be recorded and consumed independently.

4.3.4 Capability Model

The Phase 2 implementation Must enforce least-privilege access to tools and resources. The Phase 2 capability model complements GAuth's three-layer capability model (RFC 0115 §3):

- GAuth Layer 1 (core action verbs) defines what an agent is authorized to do by its mandate.
- GAuth Layer 2 (permission-based capabilities) defines scope restrictions (paths, sectors, regions).
- GAuth Layer 3 (agent/platform-based capabilities) defines platform-level permissions.
- Phase 2 capabilities define what tools and resources are operationally available to the agent, independent of mandate authorization.

An action Must satisfy both the GAuth capability model (via PEP Phase 1 evaluation) and the Phase 2 capability model (via Phase 2 evaluation) to proceed.

4.3.5 Kill Switch

The Phase 2 implementation Must provide an unconditional, immediate termination mechanism (kill switch) for any registered agent. The kill switch:

- Must be triggerable independently of GAuth mandate status (an agent May be terminated even if its mandate is still ACTIVE).
- Must take effect within 100 milliseconds of activation.
- Must produce an audit record in the unified audit trail.
- Should be triggerable via API, CLI, and dashboard interfaces.

4.3.6 Circuit Breaker

The Phase 2 implementation Should provide circuit breaker functionality to prevent cascading failures in multi-agent systems:

- Automatic isolation of agents or services exceeding configurable failure thresholds.
- State machine: CLOSED (normal) → OPEN (isolated) → HALF-OPEN (probing) → CLOSED.
- Configurable thresholds (failure rate, latency percentile, error budget consumption).
- Integration with the unified audit trail for state transition logging.

4.4 Dormancy Rules

4.4.1 Scenario-Specific Dormancy

Dormancy rules differ by scenario. In Scenario i), AGT provides only the vehicle function; GAuth implements capabilities natively. In Scenario ii), AGT operates natively; only audit and credential management are dormant.

Scenario i) Dormancy (AGT Engine):

```
| AGT Feature | Dormant | Reason | GAuth Authoritative
Equivalent |

|-------------|---------|--------|---------------------------
---|

| Policy evaluation engine | Yes | GAuth implements Phase 2
policy evaluation natively | GAuth Phase 2 policy engine |

| Execution ring enforcement | Yes | GAuth implements ring
enforcement natively | GAuth ring enforcement |

| Trust scoring (5-tier) | Yes | GAuth implements trust
scoring natively (same 5-tier model) | GAuth trust scoring |

| Kill switch | Yes | GAuth implements kill switch natively |
GAuth kill switch |

| Circuit breakers | Yes | GAuth implements circuit breakers
natively | GAuth circuit breakers |

| Flight recorder (audit logging) | Yes | GAuth enforcement
audit is authoritative | RFC 0117 §5.1 enforcement decision
audit |

| Credential management / agent registration | Yes | GAuth
mandate lifecycle is authoritative | RFC 0118 mandate CRUD |

| Credential rotation | Yes | GAuth token refresh / mandate
supersession is authoritative | RFC 0118 §6 supersession |

| PreToolUse middleware hook | **Active** | This is the
vehicle function AGT provides | N/A — this IS AGT's
contribution |
```

Scenario ii) Dormancy (AGT Bridge):

```
| AGT Feature | Dormant | Reason | GAuth Authoritative
Equivalent |
```

```
|-------------|---------|--------|-----------------------------
---|

| Policy evaluation engine | **Active** | AGT owns Phase 2
natively | N/A |

| Execution ring enforcement | **Active** | AGT owns ring
enforcement natively | N/A |

| Trust scoring (5-tier) | **Active** | AGT owns trust scoring
natively | N/A |

| Kill switch | **Active** | AGT owns kill switch natively |
N/A |

| Circuit breakers | **Active** | AGT owns circuit breakers
natively | N/A |

| Tool governance, memory, marketplace, comms | **Active** |
Complementary operational governance domain | N/A |

| Flight recorder (audit logging) | Yes | GAuth enforcement
audit is authoritative | RFC 0117 §5.1 enforcement decision
audit |

| Credential management / agent registration | Yes | GAuth
mandate lifecycle is authoritative | RFC 0118 mandate CRUD |

| Credential rotation | Yes | GAuth token refresh / mandate
supersession is authoritative | RFC 0118 §6 supersession |
```

Dormant features May emit telemetry data that is captured as supplementary fields in the unified audit record (§9), but this data is informational and Must Not influence governance decisions.

5. HOW G-AGT WORKS

5.1 Orchestrated Evaluation Model

The G-AGT evaluation model defines the normative flow for evaluating agent action requests. GAuth's PEP is the authoritative governance control plane in both scenarios. The evaluation model is structured around two complementary enforcement domains:

- **Phase 1 — Credential-Bound Delegation Enforcement** (GAuth PEP): Evaluates whether the specific authority granted via the agent's credential (mandate/PoA) permits the requested action. The 16-check PEP pipeline (RFC 0117 §9) enforces comprehensive organizational compliance rules — governance profile ceilings (CHK-03), sector restrictions (CHK-05), region restrictions (CHK-06), path

validation (CHK-07), verb permissions (CHK-08), verb constraints (CHK-09), platform permissions (CHK-10), transaction type matrix (CHK-11), decision type (CHK-12), budget (CHK-13), session limits (CHK-14), approval mode (CHK-15), and delegation chain scope narrowing (CHK-16). These rules are bound to the individual credential: the mandate specifies which sectors, regions, verbs, budget, and constraints apply to this specific agent. Phase 1 does not merely validate that a credential exists — it enforces the full organizational compliance posture encoded within that credential.

- **Phase 2 — Platform-Bound Access Control** (AGT Engine or GAuth-native): Evaluates the action against platform-level operational policies that apply independently of any specific credential. These are cross-cutting rules configured on the platform itself — policy documents (YAML, JSON, Cedar, OPA/Rego) that define organizational defaults and operational constraints applicable to all agents regardless of their individual mandates. Examples include "agents with budget > 500 EUR must execute in Ring 2 or higher," "delegated agents may not deploy to production," or "agents with Untrusted trust tier require manual approval for all actions." These rules are not derived from any credential — they are platform configuration.

An action Must satisfy both enforcement domains to proceed: the credential must authorize the action (Phase 1) AND the platform must permit it (Phase 2). This dual enforcement provides defense-in-depth: compromising the credential does not bypass platform policies, and circumventing platform policies does not override credential constraints.

Important: In Scenario i), GAuth implements both Phase 1 and Phase 2 natively — there is no compliance gap. GAuth provides complete credential-bound delegation enforcement AND platform-bound access control within a single platform. The AGT engine contributes only the vehicle function (middleware hook) for agent runtime connectivity.

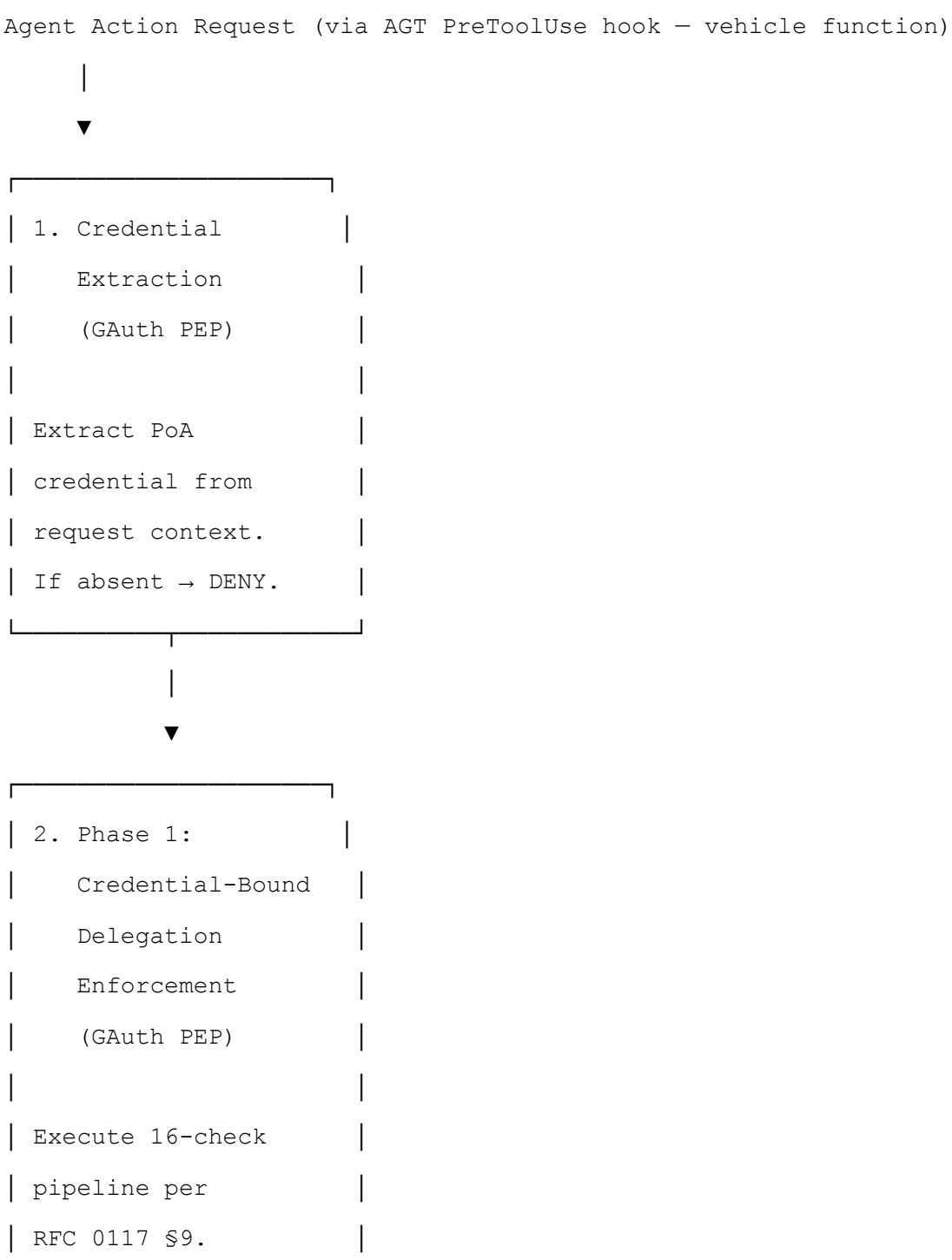
The evaluation flow differs by scenario:

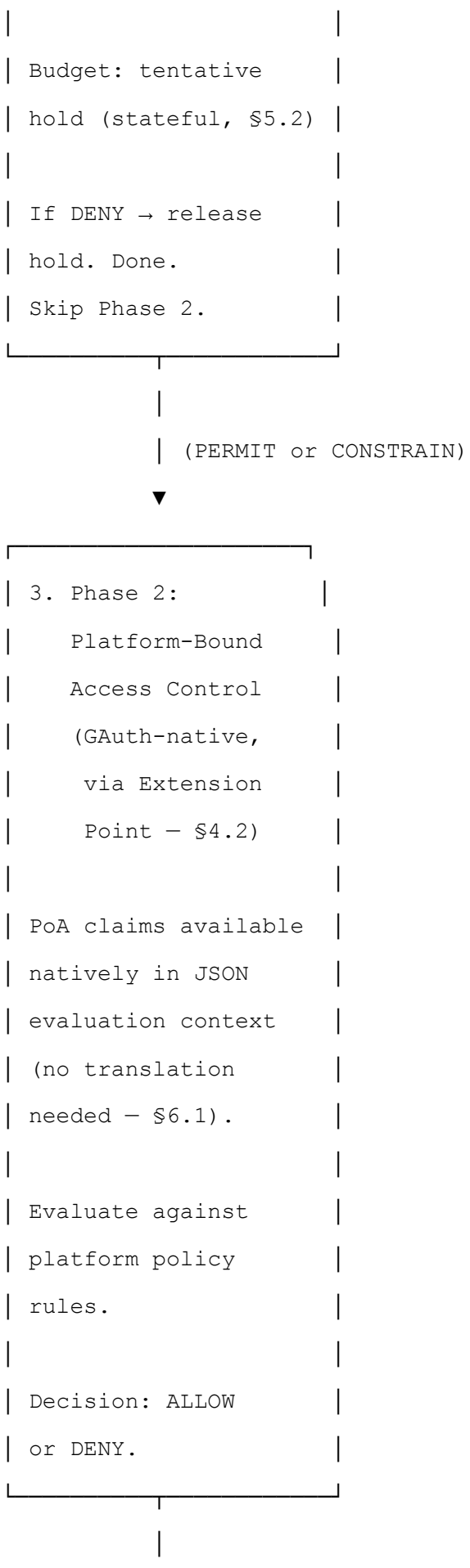
- In **Scenario i), the GAuth PEP controls the evaluation flow. The `PreToolUse` hook is the entry point: when AGT's runtime intercepts a tool call, the hook calls the GAuth PEP, which orchestrates both Phase 1 (credential-bound delegation enforcement) and Phase 2 (platform-bound access control, implemented natively by GAuth) internally. The PEP computes and enforces the final combined decision.
- In Scenario ii), AGT's runtime controls the evaluation flow. The `PreToolUse` hook fires within AGT's runtime, which calls the GAuth PEP (via the Policy Translation Bridge) for a Phase 1 (credential-bound delegation enforcement) decision. The PEP returns its decision to AGT, which then applies its own operational policies natively as Phase 2 (platform-bound access control). AGT

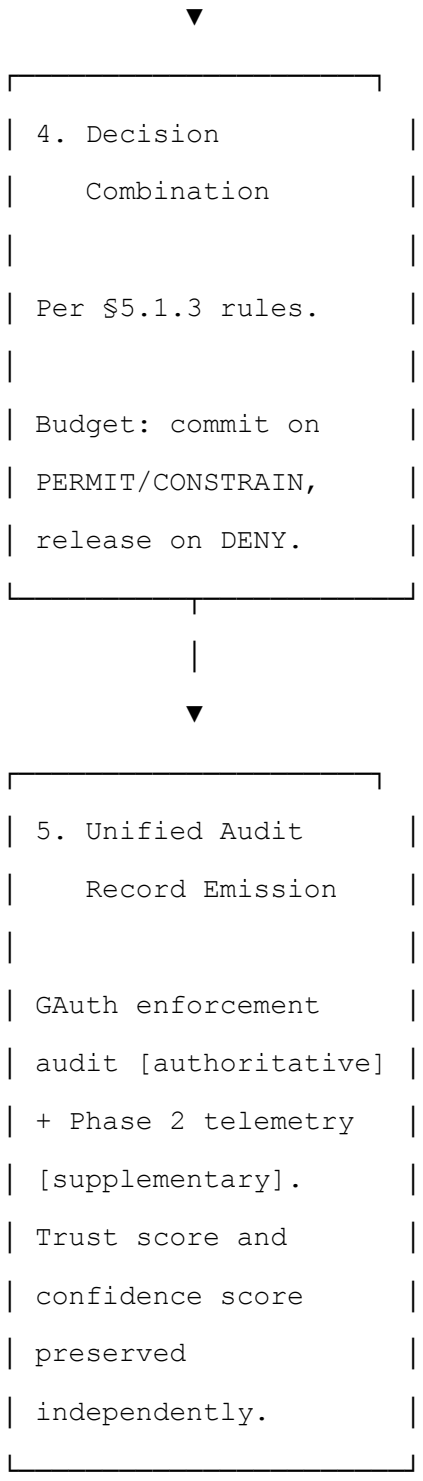
computes the final combined decision and is responsible for blocking the action if either phase returns DENY.

5.1.1 Scenario i) — AGT Engine Evaluation Flow

In Scenario i), GAuth's PEP controls the evaluation flow and invokes Phase 2 (platform-bound access control, implemented natively by GAuth) via the PEP Phase 2 extension point:




Budget: tentative hold (stateful, §5.2)
If DENY → release hold. Done. Skip Phase 2.
(PERMIT or CONSTRAIN)
3. Phase 2: Platform-Bound Access Control (GAuth-native, via Extension Point — §4.2)
PoA claims available natively in JSON evaluation context (no translation needed — §6.1).
Evaluate against platform policy rules.
Decision: ALLOW or DENY.



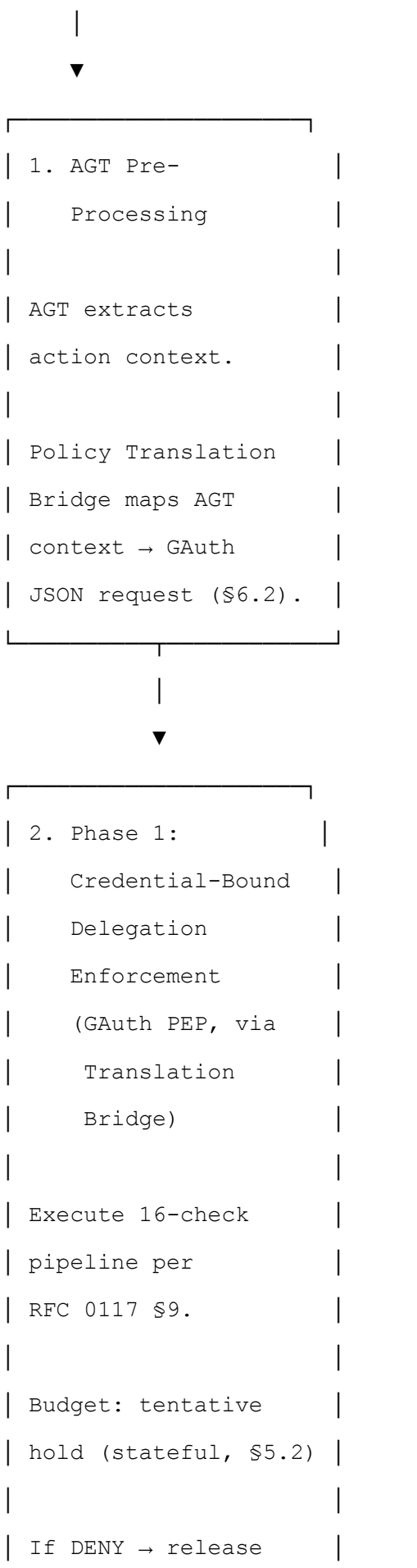

```
```

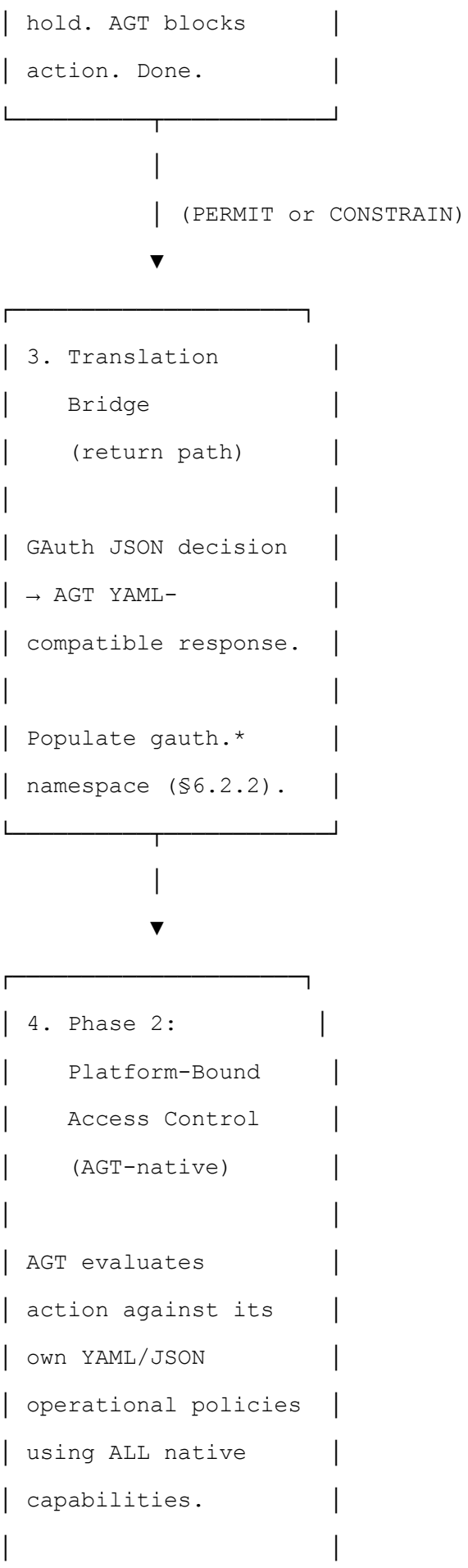
5.1.2 Scenario ii) — AGT Bridge Evaluation Flow

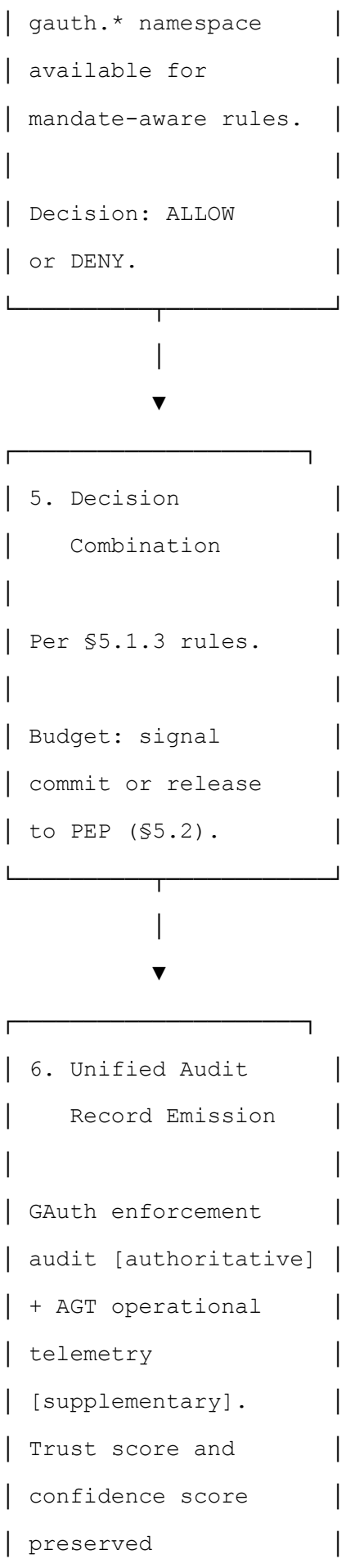
In Scenario ii), AGT's runtime controls the evaluation flow and calls GAuth's PEP for Phase 1 (credential-bound delegation enforcement) decisions:

```
```

```
Agent Action Request (via AGT PreToolUse hook — native)
```


1. AGT Pre-
Processing
AGT extracts
action context.
Policy Translation
Bridge maps AGT
context → GAuth
JSON request (§6.2).
2. Phase 1:
Credential-Bound
Delegation
Enforcement
(GAuth PEP, via
Translation
Bridge)
Execute 16-check
pipeline per
RFC 0117 §9.
Budget: tentative
hold (stateful, §5.2)
If DENY → release


hold. AGT blocks
action. Done.
(PERMIT or CONSTRAIN)
3. Translation
Bridge
(return path)
GAuth JSON decision
→ AGT YAML-
compatible response.
Populate gauth.*
namespace (§6.2.2).
4. Phase 2:
Platform-Bound
Access Control
(AGT-native)
AGT evaluates
action against its
own YAML/JSON
operational policies
using ALL native
capabilities.


gauth.* namespace
available for
mandate-aware rules.
Decision: ALLOW
or DENY.
5. Decision
Combination
Per §5.1.3 rules.
Budget: signal
commit or release
to PEP (§5.2).
6. Unified Audit
Record Emission
GAuth enforcement
audit [authoritative]
+ AGT operational
telemetry
[supplementary].
Trust score and
confidence score
preserved

```
| independently.          |
└─────────────────────────┘
```

5.1.3 Decision Combination Rules

The combined G-AGT decision follows these normative rules:

Phase 1 (GAuth PEP)	Phase 2 (AGT/GAuth-native)	Combined Decision	Budget Hold	Notes
PERMIT	ALLOW	**PERMIT**	Committed	Both phases approve.
PERMIT	DENY	**DENY**	Released	Phase 2 vetoes.
CONSTRAIN	ALLOW	**CONSTRAIN**	Committed	Action permitted with Phase 1 restrictions. Constraints unmodified by Phase 2.
CONSTRAIN	DENY	**DENY**	Released	Phase 2 vetoes despite Phase 1 conditional approval.
DENY	(skipped)	**DENY**	Released	Phase 1 DENY short-circuits. Phase 2 not evaluated.

DENY precedence rule: If either phase returns DENY, the combined decision is DENY. There is no mechanism by which Phase 2 can override a Phase 1 DENY or vice versa within the deterministic evaluation path.

Short-circuit rule: Phase 1 DENY short-circuits the evaluation — Phase 2 is never invoked. This saves Phase 2 processing and avoids unnecessary policy evaluation.

CONSTRAIN provenance rule: CONSTRAIN is produced exclusively by PEP Phase 1. Phase 2 does not produce, modify, or extend CONSTRAIN decisions. When the combined decision is CONSTRAIN, the `constrain_provenance` field in the unified audit record Must indicate "phase_1" to confirm that all constraints originated from GAuth's PEP pipeline. This ensures constraint logic remains 100% within GAuth's open core, unchanged by the G-AGT integration.

CONSTRAIN handling: When Phase 1 returns CONSTRAIN, Phase 2 evaluates the action as if Phase 1 returned PERMIT — the same policy rules are applied. If Phase 2 returns ALLOW, the combined decision is CONSTRAIN (the action proceeds with Phase 1's mandate-imposed restrictions). If Phase 2 returns DENY, the combined decision is DENY (Phase 2 vetoes the action entirely).

5.1.4 Error Handling

Fail-closed principle: G-AGT Must treat any error condition as equivalent to DENY. An agent action is only permitted when both Phase 1 (credential-bound delegation enforcement) and Phase 2 (platform-bound access control) return explicit permissive decisions.

5.2 Budget Deferral and Transactional Semantics

5.2.1 Problem Statement

In the original PEP model (RFC 0117), budget consumption occurs during Phase 1 evaluation. In a G-AGT deployment, Phase 2 (AGT) may subsequently DENY the action. Without budget deferral, the budget would be consumed for an action that was never executed — a transactional integrity violation.

5.2.2 Tentative Hold Model

When operating in stateful PEP mode (RFC 0117 §4.3), the PEP Must implement budget deferral using a tentative hold pattern:

1. Phase 1 evaluation: During CHK-08 (Budget, if applicable), the PEP places a **tentative hold** on the estimated budget amount rather than consuming it immediately. The tentative hold reserves the budget amount but does not decrement the available balance for accounting purposes. The mandate's `remaining_cents` reflects the hold (i.e., subsequent concurrent requests see the reduced available balance).

2. Phase 2 evaluation: The PEP (Scenario i) or AGT (Scenario ii) evaluates Phase 2. The tentative hold remains in place during Phase 2 evaluation.

3. Commit or release:

- If the combined decision is PERMIT or CONSTRAIN: the tentative hold is **committed** — the budget is consumed and the hold is converted to an actual budget decrement.
- If the combined decision is DENY (from either phase): the tentative hold is **released** — the reserved budget is returned to the available balance.

4. Hold expiration: Tentative holds Must have a configurable timeout (Recommended: 30 seconds). If the hold expires before commit or release (e.g., Phase 2 engine is unresponsive), the hold is released and the action is treated as DENY (fail-closed).

5.2.3 Stateless Mode

In stateless PEP mode (RFC 0117 §4.2), budget is not enforced and no hold is placed. Budget deferral applies only to stateful mode.

5.2.4 Concurrency

Tentative holds Must be concurrency-safe. Multiple concurrent Phase 1 evaluations for the same mandate Must each place independent holds. The total of all outstanding holds Must not exceed the mandate's budget ceiling. If a hold cannot be placed because outstanding holds would exceed the ceiling, the PEP Must return DENY with violation code `BUDGET_HOLD_EXCEEDED`.

5.3 Concurrency and Ordering

Phase 1 (credential-bound delegation enforcement) Must be evaluated before Phase 2 (platform-bound access control) in both scenarios. Sequential evaluation is Required because:

(a) Phase 2 depends on Phase 1 output (governance profile, PoA claims, PEP decision) to populate the evaluation context.

(b) Short-circuiting on Phase 1 DENY avoids unnecessary Phase 2 evaluation.

(c) Budget consumption in stateful PEP mode Must not be committed until Phase 2 also permits (§5.2).

6. CREDENTIAL INTEGRATION AND POLICY TRANSLATION

This section specifies how GAuth's structured JSON credential world and AGT's policy evaluation context are connected. The mechanism differs by scenario.

6.1 Scenario i) — Native JSON Context (AGT Engine)

In Scenario i), GAuth implements Phase 2 access control natively. No protocol translation is required — both Phase 1 and Phase 2 operate in a shared JSON context within the GAuth PEP. When Phase 1 returns PERMIT or CONSTRAIN, the PEP populates the Phase 2 evaluation context with PoA claims as a structured `gauth` namespace:

```json
{
```

```
  "gauth": {
    "mandate_id": "mdt_abc123",
    "governance_profile": "standard",
    "approval_mode": "supervised",
    "delegation_depth": 0,
    "max_delegation_depth": 1,
    "budget": {
      "ceiling_cents": 10000,
      "remaining_cents": 7500,
      "utilization_rate": 0.25,
      "hold_cents": 250
    },
    "temporal": {
      "issued_at": "2026-04-09T09:00:00Z",
      "expires_at": "2026-04-09T17:00:00Z",
      "remaining_seconds": 28800
    },
    "scope": {
      "allowed_verbs": [
        "urn:gauth:verb:code:file:read",
        "urn:gauth:verb:code:file:modify"
      ],
      "allowed_sectors": ["541511"],
      "allowed_regions": ["DE", "EU"],
      "allowed_paths": ["src/"],
      "denied_paths": ["src/secrets/"]
    },
    "session": {
      "max_tool_calls": 100,
      "tool_calls_used": 42,
```

```
      "max_session_duration_min": 240
    },
    "pep_decision": "PERMIT",
    "pep_constraints": [],
    "checks_passed": 16,
    "checks_failed": 0
  }
}
```

The Phase 2 policy rules reference these fields directly in JSON format. Policies May be stored in:

- **GAuth's JSON-based PoA mapping storage** — policies co-located with mandate definitions within the GAuth Management API (RFC 0118).
- **External policy servers** — dedicated policy stores that the Phase 2 engine queries, provided the policy server exposes a deterministic evaluation interface.

6.2 Scenario ii) — JSON-to-YAML Translation Bridge (AGT Bridge)

In Scenario ii), AGT runs standalone with YAML-based policy documents and calls into GAuth's PEP for Phase 1 (credential-bound delegation enforcement) decisions. The Policy Translation Bridge resolves the impedance mismatch between GAuth's JSON credential world and AGT's YAML policy language — it is an additional policy language adapter.

6.2.1 Outbound Translation (AGT → GAuth)

When AGT's `PreToolUse` hook fires, the Policy Translation Bridge translates the AGT execution context into a GAuth enforcement request, mapping AGT tool names to the GAuth verb taxonomy (§6.5):

```
| AGT YAML Context Field | GAuth JSON Enforcement Request Field | Mapping |
|------------------------|--------------------------------------|---------|
| `tool_name` | `action.verb` | Mapped to GAuth verb taxonomy: `urn:gauth:verb:{domain}:{category}:{action}` per §6.5.
```

```
Example: AGT `file_modify` →
`urn:gauth:verb:code:file:modify`. |

| `target_resource` | `action.resource` | Direct mapping. |

| `agent_id` | `agent.agent_id` | Direct mapping. |

| `agent_trust_score` | `agent.trust_score` | Direct mapping
(0–1000 integer, 5-tier). |

| `execution_ring` | `agent.execution_ring` | Direct mapping
(0–3 integer). |
```

6.2.2 Return Translation (GAuth → AGT)

The PEP returns a JSON enforcement decision. The Policy Translation Bridge translates this into AGT-compatible format and populates the `gauth.*` namespace for AGT's subsequent operational policy evaluation:

```
| GAuth JSON Decision Field | AGT YAML-Compatible Field |
Mapping |

|--------------------------|--------------------------|-------
--|

| `decision` (PERMIT/DENY/CONSTRAIN) | `gauth.pep_decision` |
Direct mapping. |

| `governance_profile` | `gauth.governance_profile` | Direct
mapping. |

| `constraints[]` | `gauth.pep_constraints` | Array of
constraint descriptors. |

| `budget.remaining_cents` | `gauth.budget.remaining_cents` |
Direct mapping. |

| `budget.hold_cents` | `gauth.budget.hold_cents` | Tentative
hold amount (§5.2). |

| `delegation_depth` | `gauth.delegation_depth` | Direct
mapping. |
```

After translation, the full `gauth` namespace (as defined in §6.1) is available in AGT's YAML policy evaluation context, enabling mandate-aware operational policy rules.

6.2.3 YAML Policy Rule Example (Scenario ii)

```yaml
name: "budget-ring-restriction"
condition:
  field: "gauth.budget.ceiling_cents"
  operator: ">"
  value: 50000
action: deny
unless:
  field: "execution_ring"
  operator: "<="
  value: 2
reason: "Agents with budget > 500 EUR must execute in Ring 2 or higher."
```

6.3 Translation Mapping Reference

The following mapping applies to the `gauth` namespace in both scenarios (native JSON in Scenario i, translated in Scenario ii):

GAuth Field	JSON Type	Condition Type	Mapping Notes
`governance_profile`	string	string (enum)	Values: "minimal", "standard", "strikt", "enterprise", "behoerde".
`approval_mode`	string	string (enum)	Values: "autonomous", "supervised", "four-eyes".
`delegation_depth`	integer	integer	Direct mapping.
`budget.remaining_cents`	integer	integer	Direct mapping.
`budget.hold_cents`	integer	integer	Tentative hold amount (§5.2). Present only in stateful mode.
`budget.utilization_rate`	float	float	Computed: `1 - (remaining_cents / ceiling_cents)`.

| `temporal.remaining_seconds` | integer | integer | Computed: `expires_at - current_time` in seconds. |

| `scope.allowed_verbs` | string[] | string[] | GAuth verb taxonomy URN format (§6.5). |

| `scope.allowed_sectors` | string[] | string[] | NAICS codes. |

| `scope.allowed_regions` | string[] | string[] | ISO 3166-1 codes. |

| `scope.allowed_paths` | string[] | string[] | Direct mapping. |

| `scope.denied_paths` | string[] | string[] | Direct mapping. |

| `session.tool_calls_used` | integer | integer | Direct mapping. |

| `pep_decision` | string | string (enum) | Values: "PERMIT", "CONSTRAIN". (DENY short-circuits before Phase 2.) |

6.4 Mandate-Aware Policy Conditions

An AGT engine participating in a G-AGT deployment (either scenario) Must support condition types that reference the `gauth` namespace. The following condition fields Must be supported:

| Condition Field | Type | Description | Example |

|---|---|---|---|

| `gauth.governance_profile` | string | Matches the governance profile of the active mandate. | `gauth.governance_profile IN ["enterprise", "behoerde"]` |

| `gauth.approval_mode` | string | Matches the approval mode. | `gauth.approval_mode == "four-eyes"` |

| `gauth.delegation_depth` | integer | Current depth in the delegation chain. 0 = direct authorization. | `gauth.delegation_depth > 1` |

| `gauth.budget.remaining_cents` | integer | Remaining budget in cents. | `gauth.budget.remaining_cents < 1000` |

| `gauth.budget.utilization_rate` | float | Budget utilization (0.0 to 1.0). | `gauth.budget.utilization_rate > 0.9` |

```
| `gauth.temporal.remaining_seconds` | integer | Time
remaining before mandate expiry. |
`gauth.temporal.remaining_seconds < 3600` |

| `gauth.scope.allowed_verbs` | string[] | Verbs the mandate
authorizes (GAuth verb taxonomy URN format). | `action.verb
NOT IN gauth.scope.allowed_verbs` |

| `gauth.session.tool_calls_used` | integer | Tool calls
consumed in current session. | `gauth.session.tool_calls_used
> 80` |

| `gauth.pep_decision` | string | GAuth PEP Phase 1 decision.
| `gauth.pep_decision == "CONSTRAIN"` |
```

6.5 GAuth Verb Taxonomy

G-AGT uses the GAuth verb namespace format defined in RFC 0117 Appendix D. Verb URNs follow the format `urn:gauth:verb:{domain}:{category}:{action}`. The following domains and verbs are reference examples from RFC 0117 Appendix D (non-normative); implementations define their own verb vocabularies within this format:

Code Agent Domain (`code`):

```
| Verb | URN |

|------|-----|

| Create file | `urn:gauth:verb:code:file:create` |

| Modify file | `urn:gauth:verb:code:file:modify` |

| Delete file | `urn:gauth:verb:code:file:delete` |

| Add dependency | `urn:gauth:verb:code:dependency:add` |

| Run command | `urn:gauth:verb:code:command:run` |

| Delegate to agent | `urn:gauth:verb:code:agent:delegate` |
```

Financial Domain (`finance`):

```
| Verb | URN |

|------|-----|

| Create transaction |
`urn:gauth:verb:finance:transaction:create` |

| Approve payment | `urn:gauth:verb:finance:payment:approve` |
```

```
| Query balance | `urn:gauth:verb:finance:balance:read` |
```

Healthcare Domain (`health`):

```
| Verb | URN |
|------|-----|
| Read patient record | `urn:gauth:verb:health:record:read` |
| Update diagnosis | `urn:gauth:verb:health:diagnosis:update`
|
| Request referral | `urn:gauth:verb:health:referral:create` |
```

Foundry Domain (`foundry`) (RFC 0116 §4.5.1):

```
| Verb | URN |
|------|-----|
| Invoke foundry agent | `urn:gauth:verb:foundry:agent:invoke`
|
| List available agents | `urn:gauth:verb:foundry:agent:list`
|
| Register agent | `urn:gauth:verb:foundry:agent:register` |
```

Implementations May define additional domain-specific verb namespaces following the same URN format. Custom domains Should not collide with the reference domains defined above.

6.5.1 AGT Tool-to-Verb Mapping

In Scenario ii), the Policy Translation Bridge Must map AGT tool names to GAuth verb taxonomy URNs. The mapping is configured per deployment and Must cover all tools registered in the AGT runtime. Unmapped tools Must be denied by default (fail-closed).

6.6 Example Mandate-Aware Policy Rules

The following rules apply to both scenarios. In Scenario i), they are expressed as JSON policy rules in GAuth's native format. In Scenario ii), they are expressed as YAML rules in AGT's format. The examples below use YAML (Scenario ii) for illustration:

Rule 1: Restrict high-budget agents to Ring 2 or higher

```yaml
name: "budget-ring-restriction"
condition:
  field: "gauth.budget.ceiling_cents"
  operator: ">"
  value: 50000
action: deny
unless:
  field: "execution_ring"
  operator: "<="
  value: 2
reason: "Agents with budget > 500 EUR must execute in Ring 2
or higher."
```

Rule 2: Block autonomous actions for four-eyes governance profile

```yaml
name: "four-eyes-no-autonomous"
condition:
  field: "gauth.approval_mode"
  operator: "=="
  value: "four-eyes"
action: deny
unless:
  field: "approval_record_present"
  operator: "=="
  value: true
reason: "Four-eyes governance profile requires dual approval
for all actions."
```

Rule 3: Reduce privilege when budget is nearly exhausted

```yaml
name: "low-budget-privilege-reduction"
condition:
  field: "gauth.budget.utilization_rate"
  operator: ">"
  value: 0.95
action: deny
unless:
  field: "trust_score"
  operator: ">="
  value: 800
reason: "Agents with > 95% budget utilization require Trusted
tier (800+) trust score."
```

Rule 4: Deny delegated agents from production deployments

```yaml
name: "no-delegated-production"
condition:
  all:
    - field: "gauth.delegation_depth"
      operator: ">"
      value: 0
    - field: "action.resource"
      operator: "matches"
      value: "prod/*"
action: deny
reason: "Delegated agents may not deploy to production."
```

```
```

7. GOVERNANCE PROFILE ALIGNMENT

This section applies to both Scenario i) and Scenario ii).

7.1 Profile-to-Ring Mapping

GAuth's five governance profiles (RFC 0115 §4) define the strictness of delegation governance. AGT's execution rings define privilege levels for access control (Scenario i) or operational governance (Scenario ii). G-AGT provides a normative mapping between these two systems, incorporating the 5-tier trust score model (§4.3.3):

```
| GAuth Governance Profile | Default AGT Ring | Approval Mode
| Max Session Duration | Delegation Permitted | Trust Score
Minimum (5-Tier) |

|------------------------|-----------------|--------------|---
------------------|--------------------|--------------------
-------|

| **minimal** | Ring 0 | autonomous | unlimited | yes
(unlimited depth) | 500 (Standard) |

| **standard** | Ring 1 | supervised | 240 min | yes (depth 1)
| 400 (Probationary) |

| **strikt** | Ring 1 | supervised | 120 min | yes (depth 1) |
500 (Standard) |

| **enterprise** | Ring 2 | supervised | 60 min | no | 600
(Standard) |

| **behoerde** | Ring 2 | four-eyes | 30 min | no | 700
(Trusted) |
```

7.2 Mapping Semantics

The profile-to-ring mapping is a default assignment, enforced by the AGT engine (Scenario ii) or by GAuth natively (Scenario i) based on the `gauth.governance_profile` claim. Implementations May override the default ring based on additional deterministic signals (trust score tier, violation counters, SLO compliance), subject to the following constraints:

- An implementation Must Not assign a **more permissive** ring than the default for the governance profile. For example, an agent with a "behoerde" mandate Must Not be assigned to Ring 0, regardless of its trust score.
- An implementation May assign a **more restrictive** ring than the default. For example, an agent with a "minimal" mandate but an Untrusted trust tier (0–299) May be assigned to Ring 2 or Ring 3.
- Ring overrides Must be logged in the unified audit trail with the reason for the override.

7.3 Approval Gate Integration

GAuth governance profiles define approval modes (autonomous, supervised, four-eyes) that Must be respected by the implementation (by GAuth natively in Scenario i, by AGT in Scenario ii):

- **autonomous**: The implementation Should not require additional approval beyond the PEP Phase 1 evaluation. The agent May execute within its ring without human intervention.
- **supervised**: The implementation Should implement a supervision mechanism that monitors agent actions and can intervene. The specific supervision UX is implementation-defined, but Must be present.
- **four-eyes**: The implementation Must enforce dual approval for all agent actions. This means at least two independent human approvers Must confirm the action before execution. The implementation Must Not permit any action under four-eyes mode without the dual approval record.

7.4 Trust Score Integration

The deterministic trust score (§4.3.3) interacts with GAuth governance profiles as follows:

- Trust scores below the minimum threshold for a governance profile's default ring Should trigger ring demotion (see §7.2). The 5-tier trust boundaries provide clear demotion thresholds (e.g., an agent with "standard" governance profile falling below 400 into Untrusted tier triggers demotion from Ring 1 to Ring 3).
- Trust score changes Should be reported to the GAuth PIP so that future PEP Phase 1 evaluations can incorporate access control trust data.
- The trust score computation Must use only the deterministic inputs defined in §4.3.3.

Trust Score vs. Confidence Score in Profile Context: The governance profile-to-ring mapping is driven exclusively by the deterministic trust score (0–1000, 5-tier). The confidence scores (0.0–1.0 float, proprietary, Type C Slot 5, thus out of scope of this RFC) do not participate in ring assignment. The confidence scores May influence the profile-to-ring mapping. The confidence scores, however, basically serve a different function. The two scores operate in separate governance domains.

8. TYPE C ADAPTER INTERFACE CONTRACTS

8.1 Overview and Strategic Role

This section defines the abstract interface contracts for Type C adapter slots — the integration points where optional proprietary services extend governance capabilities beyond the deterministic, rule-based scope of this specification.

Type C adapter interface contracts serve a critical role across all G-AGT integration scenarios:

- In **Scenario i) (AGT Engine)** and **Scenario ii) (AGT Bridge),** Type C adapters provide optional AI-enhanced governance, Web3 identity, and DNA/PQC identity capabilities on top of the open deterministic governance core.
- In the **Dynamic Scenario** (where AGT or successors implement the full GAuth specification suite), Type C adapter interface contracts become the primary differentiator for G-AGT — the exclusive gateway to Gimel's proprietary services. Even when the base GAuth specs (RFCs 0115-0118) are fully implemented by a third party, the Type C interfaces defined here remain the only specified mechanism for connecting to proprietary Gimel governance capabilities.

For this reason, Type C interface contracts are spec-level material (published under Apache 2.0), not just SDK-level. This ensures G-AGT retains clear architectural value across all scenarios.

Type C adapters are proprietary to the Gimel Foundation, require Ed25519 sealed manifest attestation, and are available at Tariff M or higher per the deployment policy matrix.

Important: Type C adapter slots are part of GAuth's connector slot model. They are architecturally distinct from the PEP Phase 2 extension point (§4.2). The AGT Engine integration does not use Type C slots. Type C adapters are optional proprietary extensions that may be present in any G-AGT deployment (either scenario) to provide AI-enhanced capabilities.

8.2 Slot 5: GovernanceAdapter (AI-Enabled Governance)

Purpose: Provides an AI/ML-enhanced governance layer on top of the deterministic PEP pipeline and deterministic Phase 2 evaluation. When present, the GovernanceAdapter receives the combined G-AGT evaluation context (PoA claims + Phase 2 context + PEP decision + deterministic trust score) and produces supplementary governance recommendations. This is the exclusive mechanism for AI-enhanced governance, adaptive policy recommendations, and ML-driven risk assessment within a G-AGT deployment.

Interface:

```
checkAccess(request: GovernanceCheckRequest) → GovernanceCheckResponse

getRecommendations(context: GovernanceContext) → GovernanceRecommendation[]

healthCheck() → AdapterHealthResult
```

Key Types:

Type	Fields
`GovernanceCheckRequest`	`requestId: string`, `operation: string`, `resource: string`, `actor: { clientId, clientType }`, `context: Record<string, unknown>`
`GovernanceCheckResponse`	`allowed: boolean`, `reason: string`, `confidenceScore: float`, `recommendations?: string[]`
`GovernanceContext`	`[key: string]: unknown` (includes merged GAuth + Phase 2 context)
`GovernanceRecommendation`	`id: string`, `recommendation: string`, `severity: string`

Confidence Score (proprietary): The GovernanceAdapter produces a confidence score (0.0–1.0 float) for implied authority assessments. This confidence score is architecturally distinct from the deterministic trust score (0–1000 integer, §4.3.3):

| Attribute | Trust Score | Confidence Score |

```
|-----------|------------|-----------------|

| **Domain** | AGT (operational) | GAuth (proprietary, Type C
Slot 5) |

| **Scale** | 0–1000 integer, 5-tier | 0.0–1.0 float |

| **Method** | Deterministic, rule-based | AI/ML-enhanced,
probabilistic |

| **Purpose** | Ring assignment, behavioral gating | Implied
authority override |

| **License** | Apache 2.0 (this spec) | Gimel Technologies
ToS |

| **Audit** | Preserved independently as `trust_score` |
Preserved independently as `confidence_score` |

| **Overwrites other?** | No | No |
```

Implied Authority Override (proprietary extension): When the deterministic evaluation (Phase 1 + Phase 2) produces a DENY decision, the GovernanceAdapter MAY override that DENY and produce a PERMIT for implied authority cases. The override applies exclusively to DENY decisions. CONSTRAIN decisions, which carry deterministic restrictions derived from the credential, Must Not be overridden or modified by the GovernanceAdapter. This override capability is the distinguishing value of the proprietary Type C Slot 5 implementation. The override is subject to the following mandatory constraints:

- The GovernanceAdapter Must produce a confidence score (0.0–1.0 float) for the implied authority assessment.
- The GovernanceAdapter Must produce a reasoning trail documenting why the implied authority was inferred.
- The override Must be accompanied by a mandatory risk disclosure that is propagated to the authority credential (PoA credential, W3C VC) issued for the action, enabling relying parties to independently verify and accept or reject the probabilistic approval.
- The unified audit record Must include: (a) the override decision, (b) the confidence score (preserved independently from the trust score), (c) the reasoning trail, and (d) the deterministic DENY that was overridden.
- The override Must Not modify, overwrite, or blend with the deterministic trust score. Both scores Must be preserved independently in the audit record.
- The override is excluded from this open specification and is not evaluated by G-AGT conformance testing (§10). It is verified under separate proprietary conformance criteria.

This override mechanism is what makes Type C Slot 5 valuable beyond the deterministic governance core: it enables governance decisions that account for implied authority — cases where rigid deterministic rule application would deny an action that a reasonable principal would have authorized, based on the overall scope and intent of the mandate.

Null Behavior: When the ai_governance slot is null, the system operates with deterministic, rule-based governance only (PEP 16-check pipeline + Phase 2 deterministic policy evaluation + deterministic trust scoring on the 5-tier model). No AI second pass, no confidence scoring, no adaptive policy recommendations, no implied authority override.

Availability: Tariff M or higher. Requires Ed25519 sealed manifest attestation.

8.3 Slot 6: Web3IdentityAdapter (Web3/DLT Integration)

Purpose: Extends the identity model with blockchain-based identity resolution and verifiable credential verification anchored on distributed ledger technology.

Interface:

```
resolveIdentity(identifier: string) → Web3Identity | null

verifyCredential(credential: unknown) → VerificationResult

healthCheck() → AdapterHealthResult
```

Key Types:

Type	Fields
`Web3Identity`	`identifier: string`, `resolved: boolean`, `[key: string]: unknown`
`VerificationResult`	`verified: boolean`, `details?: string`

Null Behavior: When web3_identity is null, standard identity resolution is used (OIDC for human identity, Ed25519/DID/SPIFFE for agent identity).

Availability: Tariff M or higher (null or attested at M, attested at L). Phase 2.

8.4 Slot 7: DNAIdentityAdapter (DNA-Based Identity / PQC)

Purpose: Extends the identity model with biometric identity binding from genomic data and post-quantum cryptographic algorithms for quantum-resistant agent authentication.

Interface:

```
resolveIdentity(identifier: string) → DNAIdentity | null

verifyBiometric(data: unknown) → VerificationResult

healthCheck() → AdapterHealthResult
```

Key Types:

Type	Fields
`DNAIdentity`	`identifier: string`, `resolved: boolean`, `[key: string]: unknown`

Null Behavior: When dna_identity is null, standard cryptographic identity is used (Ed25519 or ML-DSA-65).

Availability: Tariff L or higher only. Phase 3.

8.5 Sealed Registration and Attestation

Type C adapter registration within a G-AGT deployment follows the sealed registration protocol. Conformant implementations Must support:

- Registration of Type C adapters via the ConnectorSlotRegistry with tariff gate enforcement.
- Ed25519 manifest verification (JSON Schema, deterministic canonicalization via JCS/RFC 8785, trusted key set, temporal bounds, namespace validation).
- Adapter lifecycle state management: null → pending (registered, awaiting attestation) → active (attested and healthy) → error (health check failed).
- Null fallback behavior for all Type C slots when no adapter is installed.

8.6 G-AGT-Specific Type C Context

When Type C adapters are invoked within a G-AGT deployment, the adapter context Must include both GAuth and Phase 2 information, with trust score and confidence score preserved independently:

```json
{
  "gauth_context": {
    "mandate_id": "...",
    "governance_profile": "...",
    "pep_decision": "PERMIT",
    "delegation_depth": 0,
    "budget_hold_cents": 250
  },
  "phase2_context": {
    "scenario": "engine",
    "execution_ring": 1,
    "trust_score": 750,
    "trust_tier": "Trusted",
    "trust_score_method": "deterministic",
    "policy_evaluator_decision": "ALLOW",
    "slo_status": "healthy"
  },
  "scoring": {
    "trust_score": 750,
    "trust_score_method": "deterministic",
    "confidence_score": null,
    "confidence_score_method": null
  },
  "combined_decision": "PERMIT"
}
```

```

The `scenario` field indicates whether the deployment uses AGT Engine ("engine") or AGT Bridge ("bridge"). The `trust_score_method` field indicates whether the trust score was computed deterministically (this specification). The `confidence_score` field is null when Slot 5 is not active; when Slot 5 is active and produces a confidence score, it is populated independently (the trust score is never overwritten).

## 9. UNIFIED AUDIT TRAIL

This section applies to both Scenario i) and Scenario ii).

### 9.1 Purpose

The unified audit trail provides a single, compliance-ready audit stream for every G-AGT evaluation. GAuth's enforcement decision audit (RFC 0117 §5.1) is the authoritative record. AGT operational telemetry is merged as supplementary data.

AGT's native flight recorder (audit logging) is dormant in a G-AGT deployment (§4.4). AGT telemetry data — such as policy rules evaluated, execution ring status, and trust score at evaluation time — is captured by the unified audit record but does not constitute an independent audit trail.

Both the deterministic trust score and the proprietary confidence score (when Slot 5 is active) are preserved independently in the audit record. Neither score overwrites the other.

### 9.2 Unified Audit Record Schema

```json
{
  "record_id": "gar_<uuid>",
  "record_version": "1.1",
  "timestamp": "2026-04-09T10:00:01Z",

  "action": {
    "verb": "urn:gauth:verb:code:file:modify",
    "resource": "src/app/main.ts",
    "resource_type": "file",
    "parameters": {},
```
```

```
    "sector": "541511",
    "region": "DE"
  },

  "agent": {
    "agent_id": "agent_456",
    "service": "codeshield",
    "session_id": "sess_789",
    "human_identity": {
      "method": "oidc",
      "issuer": "https://auth.example.com",
      "principal_id": "user_123"
    },
    "agent_identity": {
      "method": "ed25519",
      "did": "did:example:agent_456",
      "verified": true
    }
  },

  "delegation": {
    "mandate_id": "mdt_abc123",
    "governance_profile": "standard",
    "approval_mode": "supervised",
    "delegation_depth": 0,
    "budget_remaining_cents": 7500,
    "mandate_status": "ACTIVE"
  },

  "pep_evaluation": {
```

```
    "decision": "PERMIT",
    "enforcement_mode": "stateful",
    "checks_performed": 16,
    "checks_passed": 16,
    "checks_failed": 0,
    "processing_time_ms": 2.3,
    "pep_version": "1.2.0",
    "violations": [],
    "constraints": [],
    "budget_hold_cents": 250,
    "budget_hold_status": "committed"
  },

  "phase2_evaluation": {
    "scenario": "engine",
    "decision": "ALLOW",
    "rules_evaluated": 47,
    "rules_matched": 0,
    "processing_time_ms": 0.8,
    "engine_version": "3.1.0",
    "violated_rules": [],
    "execution_ring": 1,
    "ring_override": null
  },

  "scoring": {
    "trust_score": 750,
    "trust_tier": "Trusted",
    "trust_score_method": "deterministic",
    "confidence_score": null,
```

```
    "confidence_score_method": null,
    "confidence_reasoning": null
  },

  "combined_decision": {
    "decision": "PERMIT",
    "total_processing_time_ms": 3.6,
    "phase_2_skipped": false,
    "constrain_provenance": null
  },

  "metadata": {
    "g_agt_version": "1.1.0",
    "correlation_id": "corr_xyz",
    "environment": "production"
  }
}
```

9.3 Required Fields

The following fields are Required in every unified audit record:

- `record_id` — unique identifier for the audit record (prefix `gar_`).
- `timestamp` — ISO 8601 timestamp of the evaluation.
- `action.verb` — the action verb evaluated (GAuth verb taxonomy URN format).
- `action.resource` — the target resource.
- `agent.agent_id` — identifier of the requesting agent.
- `delegation.mandate_id` — identifier of the PoA credential (or `null` if credential was absent).
- `pep_evaluation.decision` — GAuth PEP Phase 1 decision (PERMIT, DENY, CONSTRAIN, or `ERROR`).
- `pep_evaluation.budget_hold_status` — Budget hold status: "committed", "released", "not_applicable" (stateless mode), or "expired".

- `phase2_evaluation.scenario` — Must be "engine" (Scenario i) or "bridge" (Scenario ii).
- `phase2_evaluation.decision` — Phase 2 decision (ALLOW, DENY, `SKIPPED` if Phase 1 DENY, or `ERROR`).
- `scoring.trust_score` — Deterministic trust score (0–1000 integer) at time of evaluation.
- `scoring.trust_tier` — Trust tier name (Untrusted, Probationary, Standard, Trusted, Verified Partner).
- `scoring.trust_score_method` — Must be "deterministic" under this specification.
- `scoring.confidence_score` — Confidence score (0.0–1.0 float) when Slot 5 is active, or `null` when Slot 5 is not active.
- `scoring.confidence_score_method` — "ai_enhanced" when Slot 5 is active, or `null`.
- `combined_decision.decision` — G-AGT combined decision.
- `combined_decision.constrain_provenance` — If combined decision is CONSTRAIN, Must indicate "phase_1" to confirm constraints originated from PEP Phase 1 exclusively.

9.4 Audit Retention and Compliance

G-AGT implementations Should support configurable audit retention policies. For regulatory compliance (EU AI Act, NIST AI RMF), the following retention minimums are Recommended:

```
| Regulatory Framework | Minimum Retention | Notes |
|----------------------|-------------------|-------|
| EU AI Act (high-risk AI) | 6 months | Art. 12 — logging obligations for high-risk AI systems |
| NIST AI RMF | As defined by organizational policy | MAP 1.1, MANAGE 2.3 |
| SOC 2 Type II | 1 year | Trust Services Criteria — audit log availability |
```

Audit records Must be tamper-evident. Implementations Should use append-only storage with cryptographic chaining (hash chain or Merkle tree) to detect modification or deletion.

10. CONFORMANCE REQUIREMENTS

10.1 Conformance Levels

G-AGT defines two conformance levels, each applicable to both Scenario i) and Scenario ii):

Level	Name	Requirements
G-AGT Core	Minimum conformance for G-AGT interoperability	Must satisfy all requirements in §10.2.
G-AGT Full	Full conformance including Type C adapter support	Must satisfy all requirements in §10.2 and §10.3.

10.2 G-AGT Core Requirements

A G-AGT Core conformant implementation Must:

1. **Implement the orchestrated evaluation model** per §5.1, integrating Phase 2 access control (GAuth-native in Scenario i, AGT-native in Scenario ii) with GAuth's PEP Phase 1 enforcement flow.

2. **Support the decision combination rules** defined in §5.1.3, including the DENY precedence rule, short-circuit rule, CONSTRAIN provenance rule, and CONSTRAIN handling.

3. **Implement fail-closed error handling** per §5.1.4 for all error conditions.

4. **Support the `gauth` namespace** in the Phase 2 evaluation context — natively in JSON (Scenario i per §6.1) or via the Policy Translation Bridge (Scenario ii per §6.2).

5. **Support mandate-aware policy conditions** per §6.4, enabling policy rules to reference `gauth.*` claim fields.

6. **Implement governance profile-to-ring mapping** per §7.1, including the constraint that an implementation Must Not assign a more permissive ring than the default for the governance profile.

7. **Respect approval mode semantics** per §7.3 (autonomous, supervised, four-eyes).

8. **Emit unified audit records** per §9.2 for every G-AGT evaluation, with all Required fields per §9.3. GAuth enforcement audit Must be authoritative; Phase 2 telemetry Must be supplementary. Trust score and confidence score Must be preserved independently.

9. **Support GAuth PEP integration** — via the PEP Phase 2 extension point (Scenario i, §4.2) or HTTP binding per RFC 0117 §8 (Scenario ii).

10. **Support deterministic policy evaluation** per §4.3.1.

11. **Support at least four execution rings** per §4.3.2.

12. **Support deterministic trust scoring** on the 0–1000 scale with 5-tier boundaries (Untrusted/Probationary/Standard/Trusted/Verified Partner) per §4.3.3, using only permitted computation inputs.

13. **Provide kill switch capability** per §4.3.5.

14. **Enforce dormancy rules** per §4.4, with scenario-specific scope (§4.4.1).

15. **Support dual identity model** — human identity (GAuth OIDC) and agent identity (AGT Ed25519/DID/SPIFFE) per §2.6.

16. **Declare conformance scenario** — the implementation Must declare whether it conforms as Scenario i) (AGT Engine), Scenario ii) (AGT Bridge), or both.

17. **Support GAuth verb taxonomy** per §6.5, using `urn:gauth:verb:{domain}:{category}:{action}` format for all action verbs.

18. **Implement budget deferral** per §5.2 when operating in stateful PEP mode, including tentative hold, commit/release, and hold expiration.

10.3 G-AGT Full Requirements (in addition to Core)

A G-AGT Full conformant implementation Must additionally:

1. **Support Type C adapter slot registration** for all three Type C slots (ai_governance, web3_identity, dna_identity) per §8.

2. **Implement Ed25519 sealed manifest verification**.

3. **Support tariff gating** for Type C adapter availability per the deployment policy matrix.

4. **Implement null fallback behavior** for all Type C slots when no adapter is installed.

5. **Include G-AGT-specific context** in Type C adapter invocations per §8.6, including `scenario`, `trust_score_method`, `trust_score`, `trust_tier`, `confidence_score`, and `confidence_score_method` fields.

6. **Support circuit breaker functionality** per §4.3.6.

7. **Preserve trust score and confidence score independently** in all Type C adapter interactions — the confidence score Must Not overwrite the trust score.

10.4 Conformance Testing

G-AGT conformance is verified through test vectors analogous to the GAuth conformance test suite. A G-AGT conformance test suite Shall be published as a separate document, covering:

- Orchestrated evaluation flow for both Scenario i) and Scenario ii).
- Decision combination correctness, including CONSTRAIN provenance verification.
- Fail-closed error handling for all error conditions.
- `gauth` namespace population (native JSON for Scenario i, translated for Scenario ii).
- Mandate-aware policy rule evaluation.
- GAuth verb taxonomy compliance (§6.5).
- Governance profile-to-ring mapping enforcement, incorporating 5-tier trust boundaries.
- Approval mode semantics.
- Unified audit record schema compliance (authoritative vs. supplementary fields, scenario field, independent trust/confidence scores).
- Deterministic trust score computation (verify no AI/ML methods are used, verify 5-tier boundary classification).
- Trust score / confidence score independence (verify no overwriting or blending).
- Dormancy rule enforcement (scenario-specific per §4.4.1).
- Dual identity model validation (human identity + agent identity).
- Type C adapter registration, attestation, and null fallback (Full conformance only).
- PEP Phase 2 extension point integration (Scenario i, §4.2) and Policy Translation Bridge correctness (Scenario ii).
- Budget deferral transactional semantics (§5.2) — tentative hold, commit/release, hold expiration, concurrency.

11. SECURITY CONSIDERATIONS

11.1 Orchestrated Evaluation as Defense-in-Depth

The G-AGT orchestrated evaluation model provides defense-in-depth by requiring both credential-bound delegation enforcement (Phase 1) and platform-bound access control (Phase 2) before permitting an agent action. Compromising one governance phase does not compromise the other:

- A compromised PoA credential (GAuth Phase 1) is mitigated by Phase 2 access control policies that independently restrict agent behavior based on deterministic trust scores (5-tier), execution rings, and policy rules.
- A compromised Phase 2 policy configuration is mitigated by GAuth mandate scope restrictions (Phase 1) that independently limit what the agent is authorized to do.

In G-AGT deployments, the defense-in-depth model is further strengthened by **dual-impulse enforcement redundancy**. The same PEP 16-check pipeline is invoked by two independent impulse sources:

- **Request impulse (OAuth vehicle):** Token issuance, introspection, and refresh cycles trigger the PEP. With short refresh cycles, the enforcement context is highly granular and rich.
- **Call impulse (AGT vehicle):** Individual agent tool calls trigger the PEP via the `PreToolUse` middleware hook at the runtime level.

Both impulses invoke the identical 16-check pipeline with full enforcement authority — including verb permissions (CHK-08), verb constraints (CHK-09), transaction type matrix (CHK-11), budget (CHK-13), and all other checks. GAuth enforces PoA compliance on tool usage through the PEP pipeline regardless of impulse source; which tools an agent may use and for what purpose are governed by the mandate's authority map under both vehicles equally.

The two impulses are complementary and independent: if one impulse path is disrupted (e.g., a delayed token refresh or a bypassed middleware hook), the other continues to enforce the full PEP pipeline. This impulse redundancy directly increases enforcement reliability — a core requirement for safety-critical agent deployments.

11.2 Scenario-Specific Security Considerations

Scenario i) (AGT Engine):

- AGT provides only the vehicle function (middleware hook). GAuth implements Phase 2 access control natively within its own process boundary. The security boundary is the GAuth PEP itself — all agent action requests Must flow through the PEP, which orchestrates both Phase 1 and Phase 2.
- The AGT middleware hook Must be protected from unauthorized modification (code signing, integrity monitoring).
- Since GAuth implements Phase 2 natively, there is no network boundary between Phase 1 and Phase 2 — this reduces attack surface compared to Scenario ii.

Scenario ii) (AGT Bridge):

- The Policy Translation Bridge is a security-critical component operating across a network boundary. Implementations Must encrypt the AGT-to-PEP communication channel (mTLS Recommended).
- AGT Must verify that PEP responses originate from an authenticated, authorized GAuth PEP instance.
- The translation bridge Must validate that PoA claims originate from an authentic PEP evaluation, not from attacker-controlled input.

11.3 Policy Translation Risks (Scenario ii)

The Policy Translation Bridge (§6.2) translates between GAuth JSON and AGT YAML contexts. Implementations Must:

- Prevent claim spoofing by ensuring the `gauth` namespace in the AGT context is populated exclusively by the translation bridge from authenticated PEP output, never from external request parameters.
- Sanitize all claim values before translation to prevent policy rule injection attacks.
- Validate type correctness of translated fields (e.g., `delegation_depth` Must be a non-negative integer).

11.4 Audit Trail Integrity

The unified audit trail (§9) is the primary compliance evidence for G-AGT deployments. Implementations Must:

- Ensure audit records are tamper-evident (§9.4).
- Protect audit storage from unauthorized access.
- Ensure audit records cannot be selectively deleted without detection.
- Support forensic analysis by maintaining correlation IDs across Phase 1 and Phase 2 audit entries.
- Clearly distinguish authoritative GAuth audit data from supplementary Phase 2 telemetry.
- Preserve trust score and confidence score independently — an attacker Must not be able to overwrite trust score with confidence score or vice versa.

11.5 Fail-Closed Assurance

The fail-closed principle (§5.1.4) Must be rigorously tested and monitored:

- **Scenario i):** A common attack vector is inducing errors in the PEP Phase 2 extension point to cause GAuth to skip Phase 2 and operate with delegation-only governance. G-AGT implementations Must never fall back to Phase 1-only evaluation. If Phase 2 is unavailable, the result Must be DENY.
- **Scenario ii):** A common attack vector is preventing AGT from reaching the GAuth PEP, causing AGT to operate without delegation authority. AGT Must block all actions when the PEP is unreachable (fail-closed).

11.6 Deterministic Trust Score Integrity

The deterministic trust score (§4.3.3) Must be protected from manipulation:

- Trust score computation inputs (compliance counters, violation history) Must be stored in tamper-evident storage.
- Trust score changes Must be logged in the unified audit trail.
- The trust score computation algorithm Must be auditable — given the same inputs, any auditor Must be able to reproduce the same score.
- The trust score Must Not be influenced by the confidence score (§8.2). These are independent scoring domains with different computation methods, scales, and purposes.

11.7 Budget Deferral Security

The budget deferral mechanism (§5.2) introduces transactional semantics that Must be secured:

- Tentative holds Must be protected from manipulation — an attacker Must not be able to release a hold without the corresponding Phase 2 evaluation completing.
- Hold expiration timeouts Must be enforced — an attacker Must not be able to indefinitely extend a tentative hold to lock budget resources (denial-of-service on budget).
- Concurrent hold management Must be atomic — race conditions in hold placement or release Must not result in budget overcommitment or budget leakage.
- Audit records Must capture the full hold lifecycle (placed → committed/released/expired) for forensic analysis.

11.8 Quantum Readiness

G-AGT is designed to accommodate post-quantum cryptographic upgrades:

- GAuth Extended Tokens (RFC 0116) support algorithm agility in their signing mechanism. Current deployments use Ed25519; future deployments May use ML-DSA-65 (CRYSTALS-Dilithium) or other NIST-approved PQC algorithms.
- AGT agent identity mechanisms Should support PQC algorithm migration.
- The DNAIdentityAdapter (Slot 7, §8.4) is reserved for future PQC-based identity schemes (GiFo-RFC 0200, GiFo-RFC 0300).

11.9 Probabilistic Governance Decisions (Type C Override)

When a proprietary Type C GovernanceAdapter (Slot 5) overrides a deterministic DENY for implied authority (see §8.2), additional security considerations apply:

- **Authority disclosure requirement:** The PoA credential (JWT or W3C VC) issued for the action Must include a claim or credential attribute indicating that the authorization was granted through a probabilistic governance override, not through deterministic rule evaluation. This enables relying parties to independently assess whether they accept the probabilistic approval.
- **Confidence threshold enforcement:** Implementations Should enforce a minimum confidence threshold below which the GovernanceAdapter override is rejected and the deterministic DENY stands. The threshold is implementation-defined but Must be documented.
- **Override rate monitoring:** Implementations Should monitor the rate of GovernanceAdapter overrides relative to total evaluations. An unusually high override rate May indicate misconfigured deterministic rules or adversarial exploitation of the implied authority mechanism.
- **Relying party autonomy:** Relying parties that receive credentials with probabilistic governance disclosures retain full autonomy to reject the credential. The override mechanism does not compel acceptance — it enables a governance decision that the deterministic system would not have permitted, while maintaining full transparency.
- **Score independence:** The confidence score produced by the override Must be recorded independently from the deterministic trust score. Forensic analysis Must be able to examine both scores separately to determine whether the override was justified.
- **CONSTRAIN immunity:** The GovernanceAdapter override applies exclusively to DENY decisions. CONSTRAIN decisions carry deterministic, credential-derived restrictions and Must Not be subject to probabilistic override. This ensures that mandate-imposed constraints remain inviolable regardless of AI assessment.

RFC Cross-References

This RFC Section	References
§1 (Scope)	GiFo-RFCs 0080, 0090, 0100, 0110, 0111, 0115, 0116, 0117, 0118; IETF RFC 2119; IETF RFC 2753; OASIS XACML 3.0
§2 (Nomenclature)	GiFo-RFCs 0110, 0111, 0115, 0116, 0117, 0118; W3C DID v1.0
§3 (Why G-AGT)	GiFo-RFC 0110 (P*P prior art, §3.6); GiFo-RFC 0111; GiFo-RFC 0116 §8 (Type A adapter pattern); IETF RFC 2753 (PDP/PEP prior art); OASIS XACML 3.0 (P*P standardization); §3.8 extensibility: OPA/Rego, Cedar, Zanzibar/SpiceDB, MCP, A2A, EU AI Act, NIST AI RMF
§4 (What G-AGT Is)	GiFo-RFC 0110 (P*P architecture), 0115 §3 (three-layer capability model), 0117 §9 (PEP pipeline), 0117 §5.1 (enforcement audit)
§5 (How G-AGT Works)	GiFo-RFC 0117 §4-8 (PEP interface), §9 (evaluation pipeline), 0118 §7 (budget operations)
§6 (Credential Integration / Policy Translation)	GiFo-RFC 0116 §4-5 (Extended Token, PoA schema), 0115 §3-4 (capability model, governance profiles), 0117 Appendix D (verb taxonomy)
§7 (Governance Profile Alignment)	GiFo-RFC 0115 §4 (governance profiles)
§8 (Type C Adapter Contracts)	IETF RFC 8032 (Ed25519); IETF RFC 8785 (JCS)
§9 (Unified Audit Trail)	GiFo-RFC 0117 §5.1 (enforcement decision schema); EU AI Act Art. 12; NIST AI RMF
§10 (Conformance)	GiFo-RFC 0117 §8 (HTTP binding)
§11 (Security)	GiFo-RFCs 0200 (Gimel ID 1.0), 0300 (Gimel Authentication 1.0); IETF RFC 8032; NIST PQC

***

* * *

Disclaimer: ALL DOCUMENTS AND THE INFORMATION CONTAINED THEREIN ARE PROVIDED ON AN “AS IS” BASIS AND THE CONTRIBUTOR, THE ORGANIZATION THEY REPRESENT OR ARE SPONSORED BY (IF ANY), THE GIMEL FOUNDATION, AND ANY APPLICABLE MANAGERS OF ALTERNATE DOCUMENT STREAMS, DISCLAIM ALL WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTY THAT THE USE OF THE INFORMATION THEREIN WILL NOT INFRINGE ANY RIGHTS OR ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE.

PRODUCT NAMES, TRADEMARKS, AND REGISTERED TRADEMARKS REFERENCED IN THIS DOCUMENT ARE THE PROPERTY OF THEIR RESPECTIVE OWNERS AND ARE USED FOR IDENTIFICATION PURPOSES ONLY. NO ENDORSEMENT IS IMPLIED.

* * *